The Lain Programming Language

Language Specification

Version 1.1

August 19, 2026

Document status:	Draft — normative
Language version:	Lain 1.1
Date:	August 19, 2026

This document is the authoritative specification of the Lain programming language. It defines the syntax, semantics, and constraints that every conforming implementation shall enforce.

Contents

Foreword

This document specifies the Lain programming language, Version 1.1.

Lain is a statically-typed, compiled programming language for systems programming. It guarantees memory safety, type safety, and resource safety entirely at compile time with zero runtime overhead.

This specification is structured analogously to ISO/IEC 9899 (the C standard). Clauses 1 through 4 are administrative; Clauses 5 through 20 are normative. Annexes A and B are normative; Annexes C and D are informative.

Text marked **Constraint** describes a condition whose violation requires an implementation to issue a diagnostic message. Text marked *implementation-defined*[1] denotes behavior that is valid but whose exact result is determined by the implementation and shall be documented by it. Text marked **undefined behavior**[2] denotes a situation for which this specification imposes no requirements.

> **Note**
>
> Undefined behavior in Lain can arise only inside `unsafe` blocks or through direct C interoperability. All code in the safe fragment of Lain has defined behavior.

[1] Implementation-defined behavior:
[2] Undefined behavior:

Introduction

Design philosophy

Lain is built on five foundational pillars:

P1 — **Assembly-Speed Performance** Every language feature compiles to C code that is equivalent in performance to hand-written C. There are no hidden allocations, no garbage collector, and no reference-counting overhead.

P2 — **Zero-Cost Memory Safety** Memory safety is enforced entirely at compile time through a linear type system, a borrow checker, and value range analysis. A conforming implementation that rejects all constraint violations guarantees, for safe programs, the absence of use-after-free, double-free, null dereference, and buffer overflow at runtime.

P3 — **Static Verification** The verification properties required by this specification are decidable and polynomial in program size. No SMT solver or unbounded fixpoint iteration is required.

P4 — **Determinism** Pure functions (`func`) are referentially transparent and guaranteed to terminate. The same inputs always produce the same outputs.

P5 — **Syntactic Simplicity** Lain has no implicit copies, no hidden conversions (beyond the defined integer-widening rules), and no magic methods. Ownership is always explicit.

Relationship to C

Lain compiles to C99. The C code generated by a conforming Lain implementation is the *translation output*; this specification does not prescribe the form of that output beyond the observable behavior of the resulting program.

How to read this specification

Each normative clause is structured as follows: a prose description of the syntax (with a reference to the formal grammar in Annex A), followed by the semantics, followed by **Constraint** blocks listing every condition that requires a diagnostic message. Non-normative **Note** and **Example** environments appear throughout to aid understanding.

Chapter 1

Scope

1 This document specifies the Lain programming language. It defines the syntax, the semantic rules, and the constraints that every conforming Lain implementation shall enforce. It serves as the single authoritative reference for language users and implementors alike.

2 Lain is a statically-typed, compiled programming language designed for systems programming, embedded systems, and safety-critical software. It achieves memory safety, type safety, and resource safety entirely at compile time with zero runtime overhead.

3 This specification covers:

- lexical structure and grammar (§ 5);
- type system and type inference (§ 7);
- variable declarations, scoping, and initialization (§ 6, § 10);
- expressions, operators, and evaluation order (§ 8);
- statements and control flow (§ 9);
- functions, procedures, and purity (§ 12);
- ownership, borrowing, and the linear type system (§ 11);
- value range constraints and static verification (§ 13);
- compile-time evaluation and generics (§ 14);
- pattern matching and exhaustiveness checking (§ 15);
- module system (§ 16);
- C interoperability (§ 17);
- unsafe code boundaries (§ 18);
- memory model (§ 19);
- diagnostic messages (Annex B);
- standard library (§ 20).

4 This specification does *not* cover:

- the internal architecture of any particular compiler implementation;
- build system integration or tooling;
- IDE or editor support;
- operating system interfaces beyond what the standard library provides.

1.1 Compilation model

1 Lain uses a source-to-C transpilation model. A Lain source file (extension `.ln`) is processed by a conforming implementation to produce C99 source code, which is subsequently compiled by a C99 compiler to produce an executable. The observable behavior of the program is defined by this specification; the form of the intermediate C99 code is *implementation-defined*[1].

2 A conforming implementation shall perform the following translation phases in the following order:

1. **Lexical analysis** — source text is converted to a token stream as described in § 5.
2. **Parsing** — the token stream is reduced to an abstract syntax tree according to the grammar in Annex A.
3. **Name resolution** — identifiers are resolved to declarations as described in § 6.
4. **Type checking** — types are inferred and verified as described in § 7 through § 10.
5. **Exhaustiveness checking** — pattern matching completeness is verified as described in § 15.
6. **Use analysis (NLL)** — last-use points are computed for all variables.
7. **Linearity and borrow checking** — ownership and borrowing rules are verified as described in § 11.
8. **Value range analysis** — bounds safety and numeric constraints are verified as described in § 13.
9. **Code emission** — well-formed programs are translated to C99.

3 A program that completes all phases without a diagnostic message is **well-formed** and its translation output is defined.

> **Note**
>
> Phases 3–8 collectively form *semantic analysis*. An implementation may interleave these phases as long as the observable behavior is equivalent to executing them in the order listed.

[1] Implementation-defined behavior: the generated C99 code need not be human-readable

Chapter 2

Normative references

1 The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies.

ISO/IEC 9899:2018 (C17)
Information technology — Programming languages — C.
International Organization for Standardization, Geneva, 2018.
Referenced for: C99/C17 calling conventions (§ 17), standard integer types and their representations (§ 7), and undefined behavior categories (§ 18).

IEC 60559:2020 (IEEE 754-2019)
Floating-point arithmetic.
International Electrotechnical Commission, Geneva, 2020.
Referenced for: semantics of `f32` and `f64` arithmetic (§ 7).

2 The following documents are informative references. They are not required for conformance but provide useful background.

ISO/IEC 14882:2020 (C++20)
Programming languages — C++.
Referenced for: vocabulary and structure conventions used in this document.

The Rust Reference
The Rust Reference, `https://doc.rust-lang.org/reference/`.
Referenced for: linear type and ownership terminology.

Chapter 3

Terms, definitions, and symbols

1 For the purposes of this document, the following terms and definitions apply. Terms defined here are used with these precise meanings throughout all normative clauses. Where a term is used in its ordinary English sense rather than as a defined term, it appears without emphasis.

> **Note**
>
> Cross-references of the form *(see § N)* point to the clause where the concept is treated normatively.

3.1 General language terms

abstract syntax tree (AST)

The in-memory tree representation of a Lain program produced by the parser and consumed by semantic analysis.

behavior

External appearance or action of a program as observed by a user or another program. See also: *defined behavior*, *implementation-defined behavior*, *unspecified behavior*, *undefined behavior*.

conforming implementation

An implementation that correctly processes all programs that satisfy the rules of this specification and issues a diagnostic message for every program that violates a **Constraint** (see § 4).

conforming program

A program that uses only the syntax and semantics defined in this specification, does not rely on undefined behavior, and does not depend on implementation-defined extensions.

constraint

A requirement imposed on the source text. If a program violates a constraint, the implementation *shall* emit a diagnostic message. Constraints appear in **Constraint** environments throughout this specification.

defined behavior

Behavior that is fully and precisely specified by this document for every conforming program. Contrast with *undefined behavior* and *implementation-defined behavior*.

diagnostic message

A human-readable message issued by the implementation to report a constraint violation. Every diagnostic shall include a source location (file name, line number) and an error code of the form `[EXXX]` or `[VRA]` (see Annex B).

ill-formed program

A program that violates one or more of the syntactic or semantic rules of this specification. An ill-formed program is not a conforming program and a conforming implementation shall reject it with a diagnostic.

implementation

The system (compiler, linker, runtime) that translates a Lain source program into an executable. The reference implementation is the *lain* compiler.

implementation-defined behavior

Behavior that is valid but not fully specified by this document; the implementation shall determine the exact behavior and document it.

program

A collection of one or more Lain source files (`.ln`) that together constitute a translation unit or a set of modules.

source file

A UTF-8 encoded text file with the extension `.ln` containing Lain source text.

translation output

The C99 source code emitted by a conforming implementation for a well-formed Lain program.

undefined behavior

Behavior for which this specification imposes no requirements. A program that exhibits undefined behavior is erroneous. In the safe fragment of Lain, no operation produces undefined behavior. Undefined behavior may arise only inside `unsafe` blocks or through direct C interoperability.

unspecified behavior

Behavior for which this specification provides two or more possibilities without prescribing which shall occur. A conforming program shall not depend on unspecified behavior.

well-formed program

A program that satisfies all syntactic and semantic rules of this specification and is therefore accepted by a conforming implementation without diagnostic.

3.2 Type system terms

algebraic data type (ADT)

A composite type that is either a product type (*struct*) or a sum type (*enum*).

constrained type

A type annotated with a value range constraint of the form *T op bound*, e.g. `int >= 0` (see § 7).

integer rank

A total ordering on integer types that determines the result type of mixed-type arithmetic. The rank system is defined in § 7.

integer widening

An implicit conversion from an integer type of lower rank to one of higher rank, permitted in arithmetic and assignment contexts (see § 7).

linear type

A type whose values must be used exactly once. Struct types and resource types are linear. Primitive types (`int`, `bool`, etc.) are unrestricted (see § 11).

nominal type

A type identified by its declared name rather than its structure.

ownership mode

An annotation on a variable or parameter that specifies the kind of access the holder has: `shared` (read-only), `var` (read-write), or owned (no annotation on a linear type, transferred with `mov`) (see § 11).

primitive type

One of the built-in types: `int`, `u8`, `u16`, `u32`, `u64`, `usize`, `i8`, `i16`, `i32`, `i64`, `isize`, `f32`, `f64`, `bool`. Primitive types are unrestricted (non-linear).

structural type equivalence

Two types are structurally equivalent if they have the same structure. Lain uses nominal equivalence for user-defined types; the name is part of the type identity.

unrestricted type

A type whose values may be used any number of times without explicit consumption. All primitive types are unrestricted. Contrast with *linear type*.

3.3 Ownership and memory terms

active borrow

A borrow that is currently in the ACTIVE phase; its subject cannot be moved or mutated while the borrow is active (see § 11).

arena

A memory allocation strategy in which all allocations are made from a contiguous block; the entire block is freed at once. Lain's compiler allocates all AST nodes from arenas.

borrow

A temporary access to a value without taking ownership. A shared borrow grants read-only access; a mutable borrow grants read-write access (see § 11).

borrow phase

Either RESERVED (the borrow has been registered but is not yet active, during argument evaluation) or ACTIVE (the borrow is in force) (see § 11).

consumed variable

A linear variable whose ownership has been transferred (via `mov`). Reading or transferring a consumed variable is a constraint violation.

linear state

The tracking state of a linear variable: one of *Uninit*, *Unconsumed*, or *Consumed* (see § 11).

move

The transfer of ownership of a value from one variable or storage location to another, performed by the `mov` operator. After a move, the source is in the Consumed state.

non-lexical lifetime (NLL)

A lifetime that ends at the variable's *last use* rather than at the end of its lexical scope. Lain uses NLL to minimize the duration of borrows (see § 11).

read-write lock invariant

The aliasing invariant: at any program point, for any owned value, there exist either N shared borrows ($N \geq 0$) and no mutable borrow, or exactly one mutable borrow and no shared borrows (see § 11).

resource

A value that represents an external entity (file handle, socket, etc.) whose lifetime must be explicitly managed. Resources are modelled as linear types.

two-phase borrow

A borrow that begins in the RESERVED phase during argument evaluation and is promoted to ACTIVE after the call expression is complete (see § 11).

3.4 Control flow and purity terms

decreasing measure

An expression in a `while decreasing` statement that the implementation shall verify is non-negative when the loop condition holds and strictly decreases on every iteration (see § 9).

`func`

A pure, terminating function. A `func` shall not call a `proc` or contain an unbounded `while` loop (see § 12).

purity

The property of a `func`: its result depends only on its inputs and it has no observable side effects on any state outside its own stack frame.

`proc`

A procedure that may have side effects. A `proc` may call other `proc`s, perform I/O, and contain unbounded `while` loops (see § 12).

termination

The property that a function always reaches a `return` statement in a finite number of steps. All `func`s shall be verified to terminate by the implementation (see § 12).

3.5 Verification terms

bounds check

A runtime test that verifies an array index is within the valid range `[0, N-1]`. A conforming implementation shall eliminate bounds checks when VRA can statically prove the index is in range.

comptime

Compile-time evaluation. A `comptime` expression or function is evaluated by the compiler at compile time rather than at runtime (see § 14).

exhaustiveness

The property of a `case` expression in which every possible value of the scrutinee type is covered by at least one arm (see § 15).

in guard

A use of the `in` operator as a loop or conditional guard. An in guard of the form `idx in arr` proves to the VRA that `idx` $\in$ `[0, arr.len-1]` in the guarded body (see § 8, § 13).

interval

A pair $[\ell, u]$ representing the set of integers $\{x \mid \ell \leq x \leq u\}$. VRA represents value ranges as intervals with $-\infty$ and $+\infty$ sentinels.

monomorphization

The process by which a generic function is instantiated for each concrete set of type arguments, producing a separate, non-generic translation for each instantiation (see § 14).

return constraint

A value range constraint on the return value of a function, declared in the function signature as *T op bound* (see § 13).

value range analysis (VRA)

A static analysis that tracks integer value ranges through the program using interval arithmetic, used to eliminate bounds checks and prove constraint satisfaction (see § 13).

3.6 Module and name resolution terms

import

A directive (`import module.path`) that makes the names defined in another module available in the current module (see § 16).

module

The unit of compilation in Lain. Each source file defines one module.

qualified name

A name of the form *module.identifier* that refers to a declaration in a specific module (see § 6).

Universal Function Call Syntax (UFCS)

A syntactic sugar by which `x.f(args)` is desugared to `f(x, args)` before name resolution. UFCS allows methods to be defined as free functions (see § 12).

Chapter 4

Conformance

4.1 Conforming implementation

1 A **conforming implementation** shall:

1. accept, without diagnostic, every *conforming program*;
2. reject, with at least one diagnostic message, every program that violates any constraint stated in this specification;
3. translate accepted programs such that their observable behavior conforms to the semantic rules stated in this specification;
4. implement all features described as normative in Clauses 5 through 20;
5. document every behavior described in this specification as *implementation-defined*[1].

2 A conforming implementation may:

1. issue additional diagnostic messages (warnings) beyond those required by this specification, provided they do not cause a conforming program to be rejected;
2. accept programs through implementation-defined extensions, provided those extensions are documented and do not alter the semantics of any conforming program;
3. omit runtime bounds checks when value range analysis (§ 13) proves the access is within bounds.

> **Note**
>
> The reference implementation of Lain (*lain* compiler, version corresponding to this specification) is a conforming implementation. Where this specification marks a behavior as implementation-defined, the reference implementation's documented behavior takes precedence for programs compiled by that implementation.

4.2 Conforming program

1 A **conforming program** is a program that:

1. consists only of syntax and semantics defined in this specification;
2. does not rely on undefined behavior;

[1] Implementation-defined behavior: the exact value or behavior is implementation-defined

3. does not rely on unspecified behavior to produce a specific outcome;

4. does not depend on any implementation-defined extension.

2 A **strictly conforming program** additionally:

1. does not use any behavior documented as implementation-defined, even when that behavior is deterministic on a given implementation.

4.3 Diagnostic messages

1 A diagnostic message issued for a constraint violation shall include:

1. the source file name;

2. the line number of the offending construct;

3. the diagnostic code in the form `[EXXX]` (see Annex B);

4. a human-readable description of the violation.

2 A conforming implementation may issue additional diagnostic information (column number, source context, suggested fix) beyond the minimum required by § 4.3.

> **Note**
>
> The reference implementation additionally prints the source line and a caret (`^`) pointing to the column of the offending token.

4.4 Implementation limits

1 A conforming implementation shall support at least the following minimum translation limits. Programs that require more than these limits invoke *implementation-defined*[2].

Resource	Minimum	Clause
Identifier length (bytes)	255	§ 5
Block nesting depth	64	§ 9
Function parameters	127	§ 10
Array elements	$2^{31} - 1$	§ 7
Module import depth	32	§ 16
`case` arms per match expression	256	§ 15
Struct fields	256	§ 7
Enum variants	256	§ 7
Comptime evaluation depth	256	§ 14
Scope nesting depth	64	§ 6

[2] Implementation-defined behavior: the behavior when a translation limit is exceeded

4.5 Safe and unsafe fragments

1 The **safe fragment** of a Lain program consists of all code that does not appear inside an `unsafe` block and does not use raw pointer operations.

2 A conforming implementation shall guarantee, for any well-formed program in the safe fragment, that execution produces no:

- use of a variable after its ownership has been transferred (use-after-move);
- double transfer of ownership of the same value;
- access to an uninitialized variable;
- violation of the read-write lock invariant (§ 3.3);
- buffer overrun that has been statically proven safe by value range analysis.

3 The **unsafe fragment** comprises all code inside `unsafe` blocks. Within the unsafe fragment, the programmer is responsible for upholding the memory safety invariants listed in § 18; the implementation shall not be required to verify them.

> **Constraint**
>
> The constraints of the ownership and linearity system (§ 11) are *not* suspended inside `unsafe` blocks. Only raw pointer operations, direct ADT field access, and the bounds-check requirement are relaxed. An implementation shall continue to enforce the ownership and linearity diagnostics (`[E001]`–`[E008]` and `[E016]`) inside `unsafe` blocks.

Chapter 5

Lexical conventions

1 A Lain source file is translated in two conceptual steps: (1) the source text is decomposed into tokens as described in this clause; (2) the token stream is parsed into an abstract syntax tree according to the grammar. Implementation may combine these steps.

5.1 Source file encoding

1 A Lain source file shall be a sequence of bytes encoded in UTF-8 (Unicode Standard, Annex D). The byte order mark (U+FEFF) shall not appear in a source file. If it does, behavior is *implementation-defined*[1].

2 Multi-byte UTF-8 sequences are permitted inside string literals and comments but shall not appear in identifiers. Identifiers shall consist solely of ASCII characters (see §5.6).

3 Line endings are normalized: a carriage return (U+000D) followed immediately by a line feed (U+000A) is treated as a single line feed. A lone carriage return is treated as a line feed.

5.2 Comments

1 A *line comment* begins with `//` and extends to the first following line feed character. The line feed is not part of the comment.

2 A *block comment* begins with `/*` and ends with the nearest following `*/`. Block comments may be nested: each opening `/*` inside a block comment opens a new nesting level requiring its own closing `*/`.

3 Comments are stripped during lexical analysis and have no effect on the semantics of the enclosing program.

> **Constraint**
>
> An unterminated block comment (one for which no matching `*/` is found before the end of the source file) is *ill-formed* (`[E100]`).

5.3 Tokens

1 The result of lexical analysis is a sequence of tokens. Whitespace and comments serve solely to separate tokens. The grammar production for *token* is given in Annex A (§5.3).

2 Where the token stream is ambiguous, the lexer shall apply the *maximal munch* rule: at each position, the longest sequence of characters that forms a valid token is consumed.

> **Example**

```
..=     // one token: range-inclusive, NOT '.' followed by '.='
```

[1] Implementation-defined behavior: whether a leading BOM is silently skipped or causes a diagnostic

```
>=      // one token: greater-than-or-equal, NOT '>' followed by '='
```

3 Keywords (§5.5) take priority over identifiers: the character sequence `func` at a point where an identifier is expected is always the keyword `func`.

5.4 Whitespace

1 The whitespace characters are: space (U+0020) and horizontal tab (U+0009). They serve only to separate adjacent tokens.

2 Line feeds (U+000A), after normalization (§5.1), are significant: they act as statement terminators in contexts where a statement may follow (see §9). Multiple consecutive line feeds are treated as a single statement boundary.

3 A semicolon (`;`) may be used as a statement separator, equivalent to a line feed. Its purpose is to place more than one statement or declaration on a single line; a newline is the normal terminator, and no semicolon is needed at the end of a line. Two statements on one line without a separator are ill-formed (`[E100]`).

> **Rationale**
>
> Newlines as the primary terminator eliminate the class of bugs caused by *missing* terminators. A semicolon is available only when the programmer deliberately wants several statements on one line (e.g. `a = 1; b = 2`), making a multi-statement line an explicit, opt-in choice rather than something that can happen by accident.

5.5 Keywords

1 The following identifiers are reserved as keywords. They shall not be used as user-defined identifiers. The grammar production is *keyword* (Annex A).

Keyword	Purpose	Clause
`and`	Logical AND operator	§8
`as`	Type cast	§8
`break`	Loop break	§9
`c_include`	Foreign header include	§17
`case`	Pattern matching	§15
`comptime`	Compile-time evaluation	§14
`continue`	Loop continue	§9
`decreasing`	Bounded-loop termination measure	§9
`defer`	Deferred execution	§9
`else`	Alternative branch	§9
`extern`	Foreign function declaration	§17
`false`	Boolean literal	§5.7
`for`	Range-based iteration	§9
`func`	Pure function declaration	§12
`if`	Conditional	§9
`import`	Module import	§16
`in`	Range iteration / bounds guard	§8

(continued)

Keyword	Purpose	Clause
`mov`	Ownership transfer	§ 11
`or`	Logical OR operator	§ 8
`proc`	Procedure declaration	§ 12
`return`	Return statement	§ 9
`true`	Boolean literal	§ 5.7
`type`	Type / struct / enum declaration	§ 10
`unsafe`	Unsafe block	§ 18
`use`	Local-scope import / alias	§ 16
`var`	Mutable binding / mutable borrow	§ 10
`while`	Loop statement	§ 9

Constraint

An identifier that is identical to a keyword shall not be used in any context that requires an identifier. Such use is *ill-formed* (`[E100]`).

5.6 Identifiers

1 An identifier consists of ASCII letters, decimal digits, and underscores, and shall begin with a letter or underscore. The grammar production is *identifier* (Annex A).

2 Identifiers are case-sensitive: `foo`, `Foo`, and `FOO` are three distinct identifiers.

3 A conforming implementation shall support identifiers of at least 255 characters. The behavior when an identifier exceeds this length is *implementation-defined*[2].

Constraint

An identifier shall not be identical to a keyword (§ 5.5).

Constraint

An identifier shall begin with a letter or underscore, not a digit. A sequence of characters that begins with a digit is an integer literal (§ 5.7), not an identifier.

5.7 Literals

5.7.1 Integer literals

1 An integer literal denotes a constant integer value. Four notations are supported (grammar: *integer-literal*):

Notation	Prefix	Example
Decimal	(none)	`42`, `1_000_000`
Hexadecimal	`0x`	`0xFF`, `0x1A_3B`
Binary	`0b`	`0b1010_0101`

[2] Implementation-defined behavior: whether oversized identifiers are truncated or cause a diagnostic

Notation	Prefix	Example
Octal	`0o`	`0o755`

An underscore (`_`) may appear between any two digits of an integer literal as a visual separator. It has no effect on the value.

The type of an integer literal is inferred from context (see § 7). In the absence of type context, the default type is `int`.

> **Constraint**
>
> An integer literal whose value does not fit in the type determined by context is *ill-formed* (`[E086]`) — it is diagnosed by the same range mechanism that rejects arithmetic overflow (§ 13.10).

Example

```
var a = 255          // int, value 255
var b u8 = 255       // u8, value 255 -- OK
var c u8 = 256       // [E086]: 256 does not fit in u8
var d = 0xFF         // int, value 255
var e = 0b1111_0000  // int, value 240
var f = 1_000_000    // int, value 1000000
```

5.7.2 Floating-point literals

A floating-point literal consists of a decimal integer part, a decimal point, and a fractional part. An optional exponent part `e`*N* or `E`*N* may follow (grammar: *float-literal*).

The default type of a floating-point literal is `f64`.

Example

```
var x = 3.14       // f64
var y f32 = 1.0    // f32
var z = 2.5e3      // f64, value 2500.0
```

5.7.3 Boolean literals

The boolean literals are **`true`** and **`false`** (grammar: *bool-literal*). Both have type `bool`.

5.7.4 Character literals

A character literal is a single byte value enclosed in single quotes. It has type `u8`. Escape sequences (§ 5.7.6) are permitted.

Example

```
var a = 'A'     // u8, value 65
var n = '\n'    // u8, value 10
var z = '\0'    // u8, value 0
```

Constraint

A character literal shall contain exactly one character or one escape sequence. An empty or multi-character character literal is *ill-formed* (`[E100]`).

5.7.5 String literals

8 A string literal is a sequence of zero or more characters enclosed in double quotes. Escape sequences (§ 5.7.6) are permitted (grammar: *string-literal*).

9 The type of a string literal of N visible characters is `Fixed_u8_N+1` (an array of $N+1$ bytes, null-terminated). This type is implicitly coercible to `u8[]` (a byte slice).

10 The `.len` property of a string literal value equals N (excluding the null terminator).

Example

```
var s = "hello"    // type: Fixed_u8_6 (5 chars + null)
                   // s.len == 5
```

11 String literals are stored in the read-only data segment of the translation output. Modifying a string literal through a pointer is **undefined behavior**[3].

5.7.6 Escape sequences

12 The following escape sequences are recognized in string and character literals:

Escape	Byte value	Description
`\n`	0x0A	Line feed (newline)
`\t`	0x09	Horizontal tab
`\r`	0x0D	Carriage return
`\0`	0x00	Null byte
`\\`	0x5C	Backslash
`\"`	0x22	Double quote
`\'`	0x27	Single quote

Constraint

A backslash not followed by one of the characters in the table above is an unrecognized escape sequence and is *ill-formed* (`[E100]`).

5.8 Operators and punctuators

1 The complete set of operator and punctuator tokens is listed below. The grammar productions *operator* and *punctuator* are in Annex A.

5.8.1 Arithmetic operators

[3] Undefined behavior: the result of writing through a pointer derived from a string literal

Token	Meaning
`+`	Addition; unary identity
`-`	Subtraction; unary negation
`*`	Multiplication
`/`	Division
`%`	Modulo (remainder)

5.8.2 Comparison operators

Token	Meaning
`==`	Equal
`!=`	Not equal
`<`	Less than
`>`	Greater than
`<=`	Less than or equal
`>=`	Greater than or equal

5.8.3 Logical operators

Token	Meaning
`and`	Logical AND (short-circuit)
`or`	Logical OR (short-circuit)
`!`	Logical NOT (prefix)

5.8.4 Bitwise operators

Token	Meaning
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`«`	Left shift
`»`	Right shift

5.8.5 Assignment operators

Token	Meaning
`=`	Assignment
`+=`	Add and assign
`-=`	Subtract and assign

Token	Meaning
`*=`	Multiply and assign
`/=`	Divide and assign
`%=`	Modulo and assign
`&=`	Bitwise AND and assign
`|=`	Bitwise OR and assign
`^=`	Bitwise XOR and assign
`«=`	Left shift and assign
`»=`	Right shift and assign

5.8.6 Other operators and punctuators

Token	Meaning
`.`	Member access; module path separator
`..`	Exclusive range (half-open interval)
`->`	Function return type separator
`(`	Open parenthesis
`)`	Close parenthesis
`{`	Open brace
`}`	Close brace
`[`	Open bracket
`]`	Close bracket
`,`	List separator
`:`	Type annotation; match arm separator

2 Attributes use bracket syntax `[name]` or `[name(args)]` before a declaration (§10). The brackets `[` and `]` listed above serve both as array/slice subscripts and as attribute delimiters; the context (declaration position vs. expression position) determines the role.

5.9 Token disambiguation

1 Several token sequences have context-dependent meaning that is resolved by the parser, not the lexer. The lexer produces the same token regardless of context:

Symbol	Context 1	Context 2
`*`	Multiplication (binary)	Pointer dereference (unary prefix, only in `unsafe`)
`-`	Subtraction (binary)	Negation (unary prefix)
`.`	Member access	Module path separator
`&`	Borrowed match prefix	Bitwise AND

5.10 Integer arithmetic and overflow

1 Integer arithmetic in Lain uses two's complement representation for all signed integer types. Overflow wraps modulo 2^N where N is the bit width of the type.

2 Unsigned integer arithmetic is modular (wrapping) by definition.

3 A conforming implementation that emits C99 shall compile the translation output with `-fwrapv` or equivalent, ensuring that signed integer overflow is two's complement wrapping and not undefined behavior in the C sense.

> **Note**
>
> This differs from ISO C, where signed integer overflow is undefined behavior. In Lain, signed overflow is defined as wrapping. Programs that rely on wrapping behavior are well-formed.

4 Plain arithmetic operators (`+`, `-`, `*`, `/`) on sized integer values whose static range cannot be proven to fit the target type are *ill-formed* (`[E086]`). Explicit operators indicate the cases where overflow is intentional: `+%`, `-%`, `*%` (wrapping) and `+|`, `-|`, `*|` (saturating). See § 13.10.

Chapter 6

Basic concepts

6.1 Declarations and definitions

1 A *declaration* introduces a name into a scope and associates it with an entity: a variable, function, procedure, type, or module. Every name shall be declared before it is used, with the exception of names that are resolved by Universal Function Call Syntax (§ 12).

2 A *definition* is a declaration that also allocates storage (for variables) or provides a body (for functions and procedures). In Lain, every declaration at the local level is also a definition.

Constraint

Redeclaring a name already visible in the current or an enclosing scope — a duplicate parameter name, or a local that repeats or shadows an outer binding — is *ill-formed* (`[E013]`). Lain forbids shadowing (§ 6.2).

Constraint

Use of an undeclared identifier is *ill-formed*.

Example

```
proc f() {
    var x = 1
    var x = 2    // [E013]: redeclaration of 'x' is forbidden
}
```

6.2 Scopes

1 A *scope* is a region of program text within which a declared name can be used. Lain has the following scope levels:

Module scope Names declared at the top level of a source file are in module scope. Module-scope names are accessible throughout the module and, by default, accessible from other modules that import this module.

Function/procedure scope Parameter names are in the scope of the function or procedure body.

Block scope Names declared inside a block (`{ }`) are in the scope of that block. They are not accessible outside the enclosing block.

2 Scopes are *lexically nested*. Lain *forbids shadowing* entirely: a declaration whose name is already visible in the current scope or in any enclosing scope — a function parameter shadowed by a local, or a name redeclared in a nested block — is *ill-formed* (`[E013]`).

> **Rationale**
> Disallowing shadowing removes a class of bugs in which an inner binding silently captures a name the reader expects to denote an outer one. Every identifier in a function body therefore resolves to exactly one declaration.

3 A conforming implementation shall support at least 64 levels of nested block scope (§ 4.4).

6.3 Name resolution

1 Name resolution maps an identifier in source text to its declaration. The lookup algorithm is:

1. Search the innermost block scope.
2. If not found, search progressively wider enclosing block scopes.
3. If not found, search the function/procedure scope (parameters).
4. If not found, search the module scope.
5. If not found, search imported module scopes (in import order).
6. If not found, attempt UFCS resolution (§ 12).
7. If still not found, the identifier is undeclared and the program is *ill-formed*.

2 Because shadowing is forbidden (§ 6.2), a name resolves to at most one declaration along the scope chain: a lookup that would find a second binding of the same name is rejected at the point of the redeclaration (`[E013]`), not silently resolved by an innermost-wins rule.

6.4 Storage duration

1 Every object has a *storage duration* that determines the lifetime of its storage.

Automatic storage duration Variables declared inside a block or function body. Storage is allocated on entry to the block and deallocated on exit.

Static storage duration Variables declared at module scope. Storage persists for the lifetime of the program. *implementation-defined*[1].

Arena storage duration Objects allocated from an explicit `Arena`. Storage persists until the arena is freed as a whole.

6.5 Objects and values

1 An *object* is a region of storage interpreted according to a type. Every declared variable corresponds to an object.

2 Every variable has:

- a *declared type* (§ 7);
- an *ownership mode*: owned (default for linear types), `shared` (read-only reference), or `var` (mutable reference) (§ 11);
- a *linear state*: Uninit, Unconsumed, or Consumed (for linear types only) (§ 11).

[1] Implementation-defined behavior: whether module-scope variables are zero- initialized before `proc main` is called

6.6 Lvalues and rvalues

1 An *lvalue* is an expression that designates an object with a persistent storage location: a variable identifier, a field access on an lvalue, or an array index expression on an lvalue.
2 An *rvalue* is an expression whose value has no persistent storage location: literals, the result of arithmetic operations, or the result of a function call.
3 Assignment requires the left-hand side to be an lvalue.

> **Constraint**
>
> An assignment whose left-hand side is an rvalue is *ill-formed* (`[E100]`).

6.7 Alignment

1 Every type has a required alignment. The alignment of a type is *implementation-defined*[2]. Struct layout follows C99 struct layout rules, including padding (§ 7).
2 Stack-allocated variables are aligned to their type's required alignment. The implementation shall not insert additional padding between local variables unless required by the hardware.

[2] Implementation-defined behavior: the same as the alignment of the corresponding C99 type

Chapter 7

Types

1 Lain is statically typed. Every variable, expression, and parameter has a type determined at compile time. There is no runtime type information (RTTI). The grammar productions for types are in Annex A (*type*).

7.1 Type taxonomy

1 The types of Lain are organized as follows:

- *Primitive types*: integer, floating-point, boolean, void (§ 7.2);
- *Array types*: fixed-size, compile-time-sized (§ 7.3);
- *Slice types*: dynamically-sized views into contiguous memory, including sized slice types with static length constraints (§ 7.4);
- *Struct types*: named product types (§ 7.5);
- *Enum types*: simple enumerations (§ 7.6);
- *ADT types*: sum types with associated data (§ 7.7);
- *Pointer types*: raw memory addresses, unsafe only (§ 7.8);
- *Constrained types*: types with value range annotations (§ 7.9).

2 Types are classified as *linear* or *unrestricted*. Linear types must be consumed exactly once (see § 11). Unrestricted types may be used any number of times.

3 All primitive types are unrestricted. A struct type is linear if and only if at least one of its fields is of a linear type. This property is transitive.

7.2 Primitive types

7.2.1 Integer types

1 The integer types are listed below. Sizes marked *(platform)* are *implementation-defined*[1].

Type	Signedness	Bit width	Range	C99 type
`u8`	unsigned	8	$[0, 2^8 - 1]$	`uint8_t`
`u16`	unsigned	16	$[0, 2^{16} - 1]$	`uint16_t`

[1] Implementation-defined behavior: determined by the target platform as in the C99 standard

(continued)

Type	Signedness	Bit width	Range	C99 type
`u32`	unsigned	32	$[0, 2^{32} - 1]$	`uint32_t`
`u64`	unsigned	64	$[0, 2^{64} - 1]$	`uint64_t`
`usize`	unsigned	(platform)	$[0, 2^{N} - 1]$	`size_t`
`i8`	signed	8	$[-2^{7}, 2^{7} - 1]$	`int8_t`
`i16`	signed	16	$[-2^{15}, 2^{15} - 1]$	`int16_t`
`i32`	signed	32	$[-2^{31}, 2^{31} - 1]$	`int32_t`
`i64`	signed	64	$[-2^{63}, 2^{63} - 1]$	`int64_t`
`isize`	signed	(platform)	$[-2^{N-1}, 2^{N-1} - 1]$	`ptrdiff_t`
`int`	signed	32	$[-2^{31}, 2^{31} - 1]$	`int32_t`

2 `int` is a documented alias of `i32`, identical in every semantic respect (boundary checks, niche optimization, codegen). Use either spelling — `int` when bit-width is incidental, `i32` when explicit width matters.

3 Generalized integer types `iN`/`uN` are admitted for any $N \in [1, 64]$. Hardware-aligned widths ($N \in \{1, 8, 16, 32, 64\}$) map to standard C99 stdint types. Logical widths (other N) occupy the smallest hardware-aligned container that fits N bits when standalone; in a `[packed]` struct they occupy exactly N bits.

4 `usize` and `isize` are the platform-native unsigned and signed integer types whose width equals the pointer width. Their exact width is *implementation-defined*[2].

7.2.2 Hardware-aligned vs logical integer types

5 The `iN`/`uN` types fall into two semantic categories:

Hardware-aligned ($N \in \{1, 2, 4, 8, 16, 32, 64\}$, with the common subset being $\{8, 16, 32, 64\}$): these correspond to fundamental C99 stdint widths (`uint8_t`, `int32_t`, etc.). They are the canonical choice for C interop and for any context where exact storage size matters at the byte boundary.

Logical (other N): these are *bit-width quantities*. Their semantics are context-dependent:

- **Standalone**: storage uses the smallest hardware-aligned container fitting N bits (e.g., `u12` maps to `uint16_t`). The natural range is $[-2^{N-1}, 2^{N-1} - 1]$ signed or $[0, 2^{N} - 1]$ unsigned.
- **Inside a packed struct** (future feature, § 7.2.4): the type occupies exactly N bits of storage with bit-exact layout.

6 **Distinction from refinement.** Logical `iN`/`uN` types differ from a refinement constraint on `int`:

- `int >= 0 and <= 7` declares an `int` with a range constraint. Storage is the `int` default (`int32_t`); bit-width is not specified.
- `u3` declares a 3-bit quantity. Standalone its range is $[0, 7]$ and its storage is `uint8_t`; in a packed struct (future) it occupies exactly 3 bits.

The two forms are interchangeable when only the range matters; they differ when bit-exact storage layout matters.

[2] Implementation-defined behavior: the pointer width on the target platform

Example

```
// Logical bit-width: 12-bit quantity, container = uint16_t.
var x u12 = 4095

// Refinement constraint on int: range [0, 4095], container =
    int32_t.
var y int >= 0 and <= 4095 = 4095

// Refinement type alias: a named refinement type
    (\S\,\ref{sec:types.alias-refinement}).
type Pressure = int >= 0 and <= 1000
var p Pressure = 500
```

7.2.3 Refinement type aliases

A type alias may carry a refinement constraint that narrows the underlying type's value range:

```
type Percent = int >= 0 and <= 100
type Pressure = int >= 0 and <= 1000
type NonZero = int != 0
```

Variables declared with a refinement type alias inherit its constraint. Assignments whose source value range violates the alias constraint are *ill-formed* (`[E086]`).

Example

```
type Percent = int >= 0 and <= 100

proc main() int {
    var ok Percent = 75              // [E086 not raised]: 75
       fits [0,100]
    // var bad Percent = 150         // [E086]: 150 > 100
    return 0
}
```

7.2.4 Packed structs

The `[packed]` attribute (§ 10.1) on a struct declaration enables bit-exact memory layout. Each `iN`/`uN` field occupies exactly N bits within a single scalar container.

```
[packed]
type GpioModer {
    pin0 u2      // bit 0-1
    pin1 u2      // bit 2-3
    pin2 u2      // bit 4-5
    pin3 u2      // bit 6-7
}
// total: 8 bits -> uint8_t container, bit-exact layout
```

Layout rules:

- Fields are laid out in declaration order, starting at bit 0.
- Each field occupies exactly its declared bit width.
- Total bit width must be ≤ 64; container is the smallest of `uint8_t`, `uint16_t`, `uint32_t`, or `uint64_t` that fits the total.

11 **Access semantics**:

- Construction: `GpioModer(1, 2, 3, 0)` packs the four 2-bit fields into the container.
- Read: `r.pin1` extracts bits 2-3 via shift-and-mask.
- Write: `r.pin1 = v` updates bits 2-3, leaving other fields unchanged.

Constraint

Inside a `[packed]` struct, fields shall be `iN`/`uN` types only. Nested structs, pointers, and slices are not permitted. Violation is *ill-formed* (`[E121]`).

Constraint

The total bit width of fields in a `[packed]` struct shall not exceed 64. Otherwise *ill-formed* (`[E121]`).

Note

This feature targets embedded use cases: hardware registers, packed network protocols, codec implementations. The reference implementation emits a scalar typedef plus inline getter/setter functions for each field.

7.2.5 Integer rank and widening

12 The integer types are partially ordered by *rank*:

Rank	Types	Bit width
1	`u8`, `i8`	8
2	`u16`, `i16`	16
3	`u32`, `i32`, `int`	32
4	`u64`, `i64`	64
5	`usize`, `isize`	platform

13 **Implicit widening**: a value of a lower-rank integer type may be used where a higher-rank integer type is expected, without an explicit cast. The result type of a binary arithmetic or bitwise operation is the higher-rank operand type.

Example

```
var a u8  = 42
var b int = a          // u8 -> int: implicit widening, OK
var c = a + 1000       // u8 + int -> int (wider wins)
```

Constraint

An implicit conversion from a higher-rank integer type to a lower-rank integer type (narrowing) is *ill-formed*. Narrowing requires an explicit `as` cast (§8).

Constraint

An implicit conversion between integer types and floating-point types is *ill-formed*. Such conversions require an explicit `as` cast.

Constraint

An implicit conversion between integer types and `bool` is *ill-formed*. Such conversions require an explicit `as` cast.

7.2.6 Floating-point types

`f32` and `f64` are 32-bit and 64-bit floating-point types conforming to IEC 60559 (IEEE 754). They correspond to C's `float` and `double` respectively. `float` is a documented alias of `f32`. The default type of a floating-point literal is `f64` (§5.7.2). Arithmetic on `f32` and `f64` follows IEEE 754 semantics: overflow produces $\pm\infty$; the result of 0.0/0.0 and similar operations is NaN. These are defined behaviors in Lain.

7.2.7 Boolean type

`bool` has exactly two values: `true` and `false`. `bool` is stored as a single byte; its exact representation is *implementation-defined*[3].

Constraint

There are no implicit **value** conversions between `bool` and any integer type for general arithmetic or assignment.

The condition expression of `if` and `while` accepts either a value of type `bool` or any integer type. When an integer value is used in condition position, zero is treated as `false` and any non-zero value is treated as `true`. This relaxation is ergonomic only and does not extend to other contexts.

Example

```
var n int = read_count()
if n { ... }          // OK: integer in condition, non-zero is
    true
var b bool = n        // [E100]: not a value conversion
```

7.2.8 The void type

`void` represents the absence of a value. It is used exclusively in pointer types (`*void`) for C interoperability (see §17).

Constraint

A variable declaration with type `void` is *ill-formed*.

7.3 Array types

An array type `T[N]` denotes a contiguous sequence of N values of type `T`. N shall be a compile-time constant integer greater than zero (grammar: *array-type*). The `.len` property of an array type yields N as a compile-time constant of type `int`. Array elements are laid out contiguously in memory in increasing index order, with element 0 at the base address. The stride between elements is *implementation-defined*[4].

[3] Implementation-defined behavior: whether `true` is 1 or any non-zero value

[4] Implementation-defined behavior: equal to `sizeof(T)` in C99, including any intra-element padding

Constraint

An array size $N \leq 0$ is *ill-formed* (`[E100]`).

Constraint

Every array index expression shall satisfy $0 \leq idx < arr.len$. An access for which value range analysis cannot prove this is an out-of-bounds violation (`[E085]`).

Example

```
var arr int[5] = [1, 2, 3, 4, 5]
var x = arr[2]          // OK: 2 is statically in [0, 4]
// var y = arr[5]       // [E085]: 5 >= arr.len
```

7.4 Slice types

A slice type `T[]` is a reference to a contiguous sequence of values of type `T` whose length is determined at runtime. Its runtime representation is a *fat* pair `(len: usize, data: *T)` — a length followed by a data pointer, in that order. Because a slice carries its own length it is *not* a raw pointer, and so — unlike the thin pointer forms below — it takes *no* `*` prefix.

Note

The spelling `*T[]` is accepted as a synonym for `T[]`: the two denote the same fat-slice type and emit identical C. The `*`-prefixed forms that carry no runtime length (`*T[N]`, `*T[:S]`, `*T`) are *thin* views; the bare bracket forms (`T[]`, `T[n]`) are fat. `T[]` is the canonical slice spelling.

The full family of reference-to-array types:

Syntax	Semantics	Runtime cost
`T[N]`	inline array (N elements owned)	N × sizeof(T)
`T[]`	fat slice, runtime length	usize + ptr
`*T[N]`	thin view, compile-time length	ptr only
`*T[:S]`	thin view, sentinel-terminated	ptr only
`*T`	raw pointer (no length)	ptr only

A slice has two properties accessible via dot notation:

- `.len`: the number of elements, type `usize`;
- `.data`: a pointer to the first element, type `*T`.

A string literal of N characters has static type `+1Fixed_u8_N` (a null-terminated fixed-size array of $N+1$ bytes) and is implicitly coercible to `u8[]`.

7.4.1 Sized slice types

A *sized slice type* is a slice type annotated with a size constraint on its length. The constraint is part of the parameter or variable declaration, not of the nominal type itself. The syntax is:

Syntax	Meaning
`T[]`	unsized dynamic slice (no length constraint)
`T[n]`	length exactly `n`; n is a size expression
`T[>= n]`	length $\geq$ `n`; n is a size expression
`T[> n]`	length $>$ `n`; n is a size expression

A *size expression* (*size_expr*) may be any of:

- an integer literal (e.g., `3`, `0`);
- an identifier that names a numeric variable in scope (e.g., `n`);
- a member expression of the form `p.len`, where `p` is a slice or array parameter in scope (e.g., `a.len`, `src.len`);
- a binary arithmetic expression over the above (e.g., `a.len + b.len`, `src.len - 1`).

Example

```
// Exact-length constraint: out must have length == a.len +
    b.len
func concat(a i32[], b i32[], out i32[a.len + b.len]) { ... }

// Lower-bound constraint: arr must have at least n elements
func first(arr i32[> 0]) i32 { return arr[0] }

// Derived constraint using member .len
func slide(src i32[], out i32[src.len - 1]) { ... }
```

Constraint semantics.

- `T[expr]`: the value range analysis (VRA, § 13) records the constraint `len == expr` and uses it to prove subsequent bounds checks inside the function body.
- `T[>= expr]`: VRA records `len >= expr`.
- `T[> expr]`: VRA records `len > expr`, i.e. `len >= expr + 1`.

Call-site verification. At every call site, the compiler verifies that the argument satisfies the formal parameter's size constraint (**[E087]**); see § 13.12.

C ABI decomposition (Fase 7). When a function parameter has a sized or unsized dynamic slice type, the compiler decomposes it into a pair of C parameters:

- a length parameter `size_t __len_NAME`;
- a data pointer `const T*` (shared) or `T* restrict` (mutable).

If the parameter's size expression is an arithmetic expression over other parameters' lengths (e.g., `out i32[a.len + b.len]`), no `__len_out` parameter is emitted; the length is re-derived from `__len_a + __len_b` at every use site.

Example

```
// Source:
proc vadd(var out i32[], a i32[out.len], b i32[out.len]) { ... }

// C emission:
```

```
// void vadd(size_t __len_out, int32_t * restrict out,
//           const int32_t* a, const int32_t* b);
```

7.5 Struct types

A struct type groups named fields into a single named product type. The grammar production is *struct-declaration* (Annex A).

Field order is the declaration order. The memory layout of a struct is *implementation-defined*[5].

A struct type S is linear if at least one field of S has a linear type (transitively).

Constraint

Q-008. Every field of a struct or enum variant whose declared type is linear shall be annotated `mov`. Omitting `mov` on a linear field is *ill-formed* (`[E083]`). This requirement makes linearity locally visible in the declaration: a reader of a struct definition does not need to inspect the field's type definition to know whether the struct is linear.

Example

```
type Handle {
    mov ptr *int              // explicit ‘mov‘ on linear field
}

type Session {
    mov h Handle              // OK: Handle is linear, mov
        required
    name u8[]                 // unrestricted field, no mov needed
}

// Ill-formed: missing mov on a linear field
type Bad {
    h Handle                  // [E083]
}
```

Struct construction: all fields shall be provided in declaration order. The type of each argument shall be compatible with the declared field type (§ 7.10).

Constraint

A struct constructor expression with a number of arguments different from the number of fields is *ill-formed* (`[E100]`).

Constraint

Writing to a field of a struct variable requires the variable to be declared as `var` (mutable). Writing to an immutable binding's field is *ill-formed*.

Constraint

Using `==` or `!=` on a struct type is *ill-formed*. Field-by-field comparison must be performed explicitly.

[5] Implementation-defined behavior: equal to the C99 struct layout, including compiler-inserted padding for alignment

7.6 Enum types

1 An enum type is a type with a finite, named set of values. Each variant carries no associated data. The grammar production is *enum-declaration* (Annex A).
2 The underlying representation of an enum type is *implementation-defined*[6].
3 Enum values are accessed as `TypeName.VariantName`.

> **Constraint**
> Using `==` or `!=` to compare enum values is *ill-formed*. Enum values shall be compared using `case` pattern matching (§15).

> **Note**
> The constraint on `==` for enums reflects that the underlying tag representation is implementation-defined. Pattern matching is always sound.

7.7 Algebraic data types (ADTs)

1 An ADT is a sum type in which each variant may carry associated fields. A type declaration is classified as an ADT if at least one variant has fields. The grammar production is *enum-declaration* (Annex A, using the record-variant form).
2 ADTs use a niche-based representation whenever a zero-cost encoding is available; otherwise they fall back to a tagged union (a tag field of implementation-defined integer type plus a union of variant payload structs).

> **Note**
> **D-Niche aggressive multi-slot.** When the variant payload admits a *niche*—a bit pattern that cannot occur in any valid value of the payload type—that niche is used to encode tag-less variants. Niche sources, in priority order: zero-page pointer slots for pointers/slices; VRA-proven unused values for constrained integers; bit patterns outside the declared variants for `bool` or small enums; niche of any recursive field. See §19.9 for the full algorithm.
>
> If no niche or only a partial niche can be derived, the compiler emits `[W120]` (best-effort warning, never an error) and falls back to a tag+union representation. The programmer may resolve the warning by constraining the payload, by changing to a payload type with larger sentinel space, or by accepting the tag byte. See §19.9 for the full layout algorithm and target dependencies.

3 ADT values are constructed as `TypeName.VariantName(args)` for variants with fields, or `TypeName.VariantName` for unit variants.
4 ADT values are destructured via `case` pattern matching (§15).

7.8 Pointer types

1 A pointer type `*T` or `const *T` is a raw memory address. Pointer operations require an `unsafe` block (§18).
2 The ownership modes for pointer types are:

[6] Implementation-defined behavior: an integer type large enough to hold all variant tags; the reference implementation uses `int`

Lain syntax	Mode	C99 type	Linear?
`*T`	shared (read-only)	`const T*`	no
`var *T`	mutable	`T*`	no
`mov *T`	owned	`T*`	yes

Constraint

Dereferencing a pointer or taking the address of a variable shall occur only inside an `unsafe` block. Pointer dereference in safe code is *ill-formed* (`[E060]`).

7.9 Constrained types

1 A constrained type is a type annotated with a value range constraint:

$$T \ op \ bound$$

where $op \in \{>=, <=, >, <, ==\}$ and *bound* is an integer literal.

2 Constrained types are used in function return types and parameter types to declare value range invariants that the value range analysis (§ 13) shall enforce.

Example

```
func nonzero_divisor() int >= 1 {
    return 42   // VRA verifies: 42 >= 1 holds
}
```

3 The constraint is part of the function signature, not the type system's nominal identity. Two constrained types with the same base type are assignment-compatible.

7.10 Type compatibility and equivalence

1 Lain uses **nominal type equivalence**: two types are the same if and only if they have the same declared name.

2 Two types T_1 and T_2 are *compatible* for an expression context if:

- $T_1 = T_2$ (same nominal type), or
- T_1 is an integer type of lower rank than T_2 (implicit widening; § 7.2.5).

Constraint

An expression of type T_1 used in a context requiring type T_2, where T_1 and T_2 are not compatible, is *ill-formed*.

3 The operators `==` and `!=` are defined for:

- primitive numeric types;
- `bool`;
- pointer types.

They are not defined for struct types, array types, or enum types.

Chapter 8

Expressions

1 An expression is a syntactic construct that evaluates to a value. Every expression has a type, determined statically at compile time. The grammar production for *expression* is in Annex A.

2 Expressions are evaluated strictly left-to-right: for any binary operator *a op b* or function call `f(a, b)`, the left operand (or earlier argument) is fully evaluated before the right operand (or later argument).

8.1 Operator precedence and associativity

1 The following table lists all binary operators in order of decreasing precedence. All operators listed on the same row have equal precedence.

Prec.	Operators	Assoc.	Result type
1 (high)	`() [] .`	left	(context-dependent)
2	unary `- ! mov`	right	(operand type)
3	`as`	left	target type
4	`* / %`	left	wider operand type
5	`+ -`	left	wider operand type
6	`<< >>`	left	left operand type
7	`&`	left	wider operand type
8	`^`	left	wider operand type
9	`|`	left	wider operand type
10	`< > <= >= in`	left	`bool`
11	`== !=`	left	`bool`
12	`and`	left	`bool`
13 (low)	`or`	left	`bool`

2 Parentheses override precedence.

8.2 Primary expressions

1 A primary expression is one of: an identifier, a literal, a parenthesized expression, a block expression, an `if` expression, a `case` expression, a constructor expression, or a `comptime` expression.

2 **Identifier expression**: evaluates to the current value of the named variable.

> **Constraint**
>
> The identifier shall be declared in scope; use of an undeclared identifier is *ill-formed* (§ 6.3).

> **Constraint**
>
> The identifier shall have been initialized before use (`[E005]`).

8.3 Arithmetic operators

1 The binary arithmetic operators are `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and `%` (remainder). The result type is the wider of the two operand types (§ 7.2.5).
2 Integer division truncates toward zero. `f32`/`f64` division follows IEEE 754.

> **Constraint**
>
> Integer division or remainder where the divisor is statically provable to be zero (via value range analysis, § 13) is *ill-formed* (`[E015]`).

> **Constraint**
>
> `%` shall not be applied to floating-point operands.

3 The unary `-` operator negates its operand. For unsigned integer types the result wraps modulo 2^N.

8.4 Comparison operators

1 The comparison operators `==`, `!=`, `<`, `>`, `<=`, `>=` compare two values and yield `bool`.

> **Constraint**
>
> Operands of a comparison operator shall have compatible types (§ 7.10).

> **Constraint**
>
> `==` and `!=` shall not be applied to struct types, array types, or enum types (§ 7.6).

8.5 Logical operators

1 `and` and `or` are short-circuit operators: the right operand is not evaluated if the result is determined by the left operand alone. `!` is prefix unary negation. All yield `bool`.

> **Constraint**
>
> Operands of `and`, `or`, and `!` shall have type `bool`.

8.6 Bitwise operators

1 The bitwise operators `&`, `|`, `^`, `<<`, `>>` operate on integer operands. The result type is the wider operand type for `&`, `|`, `^`; for shift operators, the result type is the left operand type.

> **Constraint**
>
> Operands of bitwise operators shall be integer types.

Constraint

The shift amount (right operand of `<<` and `>>`) shall satisfy $0 \leq \mathit{shift} < N$ where N is the bit width of the left operand. A shift amount that value range analysis cannot prove to be in range is *ill-formed* (`[E086]`) — the boundary mechanism (§13.10) that also rejects arithmetic overflow.

8.7 The `in` operator

1 The expression `idx in arr` evaluates to `true` if and only if $0 \leq \mathit{idx} < \mathit{arr.len}$. The operand *arr* shall be an array or slice type; *idx* shall be an integer type.

2 When `idx in arr` appears as the condition of a `while` or `if` statement (or as the leftmost operand of an `and`-chain condition), it establishes an **in guard**: inside the body of that statement, the implementation shall accept `arr[idx]` without requiring further bounds verification.

3 In-guards propagate through `and`-chains: if the left operand of `and` establishes an in-guard, that guard is active when evaluating the right operand.

Example

```
// In-guard: arr[i] is safe without explicit VRA
while i in arr {
    var x = arr[i]    // OK: in-guarded
    i = i + 1
}

// And-chain propagation:
while l.pos in l.src and (l.src[l.pos] as int) != '"' {
    l.pos = l.pos + 1
}
```

Constraint

An in-guard of the form `idx in arr` covers only `arr[idx]` exactly. Accesses `arr[idx + k]` for any $k \neq 0$ are not covered by this guard and require separate verification or an `unsafe` block.

8.8 Cast expression (`as`)

1 The expression `expr as T` explicitly converts the value of *expr* to type `T`. The following casts are permitted:

From	To	Semantics
integer	integer	Truncation (narrowing) or extension (widening)
integer	float	Nearest representable value (IEEE 754)
float	integer	Truncation toward zero
float	float	Precision change
pointer	pointer	Reinterpretation (`unsafe` required)
integer	pointer	Reinterpretation (`unsafe` required)
pointer	integer	Reinterpretation (`unsafe` required)

> **Constraint**
>
> Casts between pointer types, between a pointer and an integer, or between any type and a pointer shall occur only inside an `unsafe` block. Such a cast outside `unsafe` is *ill-formed* (`[E012]`).

> **Constraint**
>
> A cast between non-numeric, non-pointer types (e.g., struct to integer) is *ill-formed*.

8.9 Move expression (`mov`)

1 `mov expr` transfers ownership of the value produced by *expr*. The source is placed in the Consumed state; any subsequent use of the source variable is *ill-formed* (`[E001]`). See § 11.

8.10 Borrows at call sites

1 Lain has *no* prefix borrow operator (no `&`, no `ref`, no `mut` expression form). A value is borrowed simply by passing it to a function or procedure; the borrow *mode* is written at the call site, matching the parameter's declared mode: nothing for a shared (read-only) borrow, `var` for a mutable borrow, and `mov` to transfer ownership (§ 8.9). See the call-site mode table in § 8.14 and the ownership rules in § 11.

8.11 Field access

1 The expression `expr.field` accesses the named field of a struct or ADT value, or a property of an array or slice (`.len`, `.data`).

> **Constraint**
>
> The field name shall exist on the type of *expr*.

> **Constraint**
>
> Writing to a field (`expr.field = v`) requires *expr* to be a mutable lvalue.

8.12 Index expression

1 The expression `arr[idx]` accesses the element of array or slice *arr* at integer index *idx*.

> **Constraint**
>
> The value range analysis shall prove $0 \leq idx < arr.len$ at every index expression. An index expression for which this cannot be proved is *ill-formed* (`[E085]`). An `in` guard (§ 8.7) or an `unsafe` block suppresses this requirement.

8.13 Range index expression (sub-slicing)

1 The expression `arr[a..b]` produces a *sub-slice*: a slice covering elements at indices $a, a+1, \ldots, b-1$ of `arr`. The range is **half-open**: the element at index b is excluded. The length of the result is $b - a$.

2 The inferred type of `arr[a..b]` is `T[b-a]`, a sized slice type (§ 7.4.1) whose length constraint records the exact length of the result.

Constraint

The start index `a` shall satisfy $a \geq 0$. A start index for which VRA cannot prove non-negativity is *ill-formed* (`[E085]`).

Constraint

The end index `b` shall satisfy $b \leq arr.len$. An end index for which VRA cannot prove this is *ill-formed* (`[E085]`).

Constraint

When both `a` and `b` are integer literals, the compiler additionally verifies $a \leq b$. If $a > b$, the program is *ill-formed* (`[E087]`).

Example

```
var xs i32[5] = [10, 20, 30, 40, 50]
var s = xs[1..4]       // type i32[3]; covers indices 1, 2, 3
var e = xs[3..3]       // OK: empty sub-slice, length 0
// var bad = xs[5..2] // [E087]: 5 > 2 (literal ordering
    violated)
```

8.14 Function call

1 A function call expression `f(args)` evaluates the arguments and transfers control to the callee. Arguments are evaluated left-to-right.

2 Argument ownership modes shall be given explicitly at the call site:

Syntax	Meaning
`f(x)`	shared borrow (default; parameter declared with no mode keyword)
`f(var x)`	mutable borrow (parameter declared `var`)
`f(mov x)`	ownership transfer (parameter declared `mov`)

Constraint

The mode annotation at the call site shall match the declared parameter mode. A mismatch is *ill-formed*.

Constraint

The number of arguments shall equal the number of parameters (unless the callee is a variadic C function declared with `...`).

3 **Two-phase borrows**: during argument evaluation, borrows of the receiver of a method call are placed in the RESERVED phase and promoted to ACTIVE after all arguments are evaluated (see § 11).

8.15 Block expression

1 A block expression `{ stmts... expr }` evaluates each statement in order, then evaluates the final expression, whose value is the value of the block. If the final expression is absent, the block has no value.

8.16 If expression

1 An `if` expression of the form `if cond { e1 } else { e2 }` evaluates *cond*; if true, evaluates e_1; otherwise, evaluates e_2.

> **Constraint**
>
> The types of e_1 and e_2 shall be compatible. If they are not, the expression is *ill-formed*.

> **Constraint**
>
> *cond* shall have type `bool`.

8.17 Match expression (`case`)

1 A `case` expression (*match-expression*) selects one of several alternatives based on pattern matching. All arms shall produce values of compatible types; the type of the `case` expression is that common type. See § 15 for full exhaustiveness rules.

8.18 Constructor expression

1 A constructor expression builds a value of a struct or ADT type by supplying all field values. See § 7.5 and § 7.7 for constructor rules.

8.19 Compound assignment

1 Compound assignment `x op= e` is syntactic sugar for `x = x op e`. The left-hand side shall be a mutable lvalue. All compound forms of the arithmetic and bitwise operators are supported (see § 5.8).

Chapter 9

Statements

A statement is a unit of execution. Statements are terminated by a line feed or a semicolon (§5.4); the semicolon is used to place several statements on one line. The grammar production for *statement* is in Annex A.

9.1 Block statement

A block statement `{ stmts... }` creates a new scope (§6.2) and executes each contained statement in order. Names declared inside a block are not accessible outside it.

9.2 Immutable declaration

`name [Type] = expr` declares and initializes an immutable binding. The type may be omitted (inferred from the initializer). The binding cannot be reassigned after initialization.

```
x i32 = 42            // explicit type
y = 10                // type inferred (i32)
p Point = Point(1, 2)
```

> **Constraint**
>
> An immutable declaration requires an initializer expression: an immutable binding must have a definite value at its point of declaration.

> **Constraint**
>
> When a type annotation is present, the type of the initializer shall be compatible with the declared type (§7.10).

> **Note**
>
> When the type is omitted, the distinction between declaration and reassignment is resolved by scope: `x = 5` where `x` is not yet declared creates a new immutable binding; where `x` is already declared as `var`, it is a reassignment.

9.3 Mutable variable declaration

`var name Type = expr` declares a mutable variable. Mutable variables can be reassigned. The type may be omitted if it can be inferred from the initializer.

```
var counter i32 = 0
counter = counter + 1        // OK: var is mutable
```

```
var buf u8[256]                  // OK: type present; unassigned until written
                                 // (reading before a write is [E005])
```

Constraint

A mutable declaration shall include either an explicit type annotation or an initializer expression. A declaration with neither is *ill-formed* (`[E100]`).

Constraint

When both a type annotation and an initializer are present, the type of the initializer shall be compatible with the declared type (§7.10).

9.4 Assignment statement

An assignment statement `lvalue = expr` stores the value of *expr* into the storage designated by *lvalue*. An assignment target without a type annotation is always a reassignment, never a declaration.

Constraint

The left-hand side shall be a mutable lvalue (§6.6). Assigning to an immutable binding is *ill-formed* (`[E009]`).

Constraint

Overwriting a linear variable whose current value has not been consumed would leak that value. Such an assignment is *ill-formed* (`[E003]`).

Note

Assigning to a linear variable after its previous value has been consumed (e.g., loop reassign-and-consume pattern) is permitted. The implementation resets the variable's linear state to Unconsumed after a legal overwrite.

Example

```
import std.fs

proc main() int {
    var f = open_file("data.txt", "r")
    close_file(mov f)                // f is now Consumed
    f = open_file("data.txt", "r")   // OK: overwriting Consumed state
    close_file(mov f)
    return 0
}
```

9.5 Expression statement

An expression followed by a statement terminator is an expression statement. The value of the expression is discarded.

9.6 If statement

1 `if cond { body } [else if cond { body }]... [else { body }]` evaluates *cond*; if true, executes the then-body; otherwise, tests the `else if` clauses in order; if none matches, executes the `else` body (if present).

> **Constraint**
>
> *cond* shall have type `bool`.

2 **Range refinement**: inside the then-branch of `if x < N { }`, the value range analysis (§13) knows that $x \in (-\infty, N-1]$; inside the else-branch, $x \in [N, +\infty)$.

3 **In-guard**: when *cond* contains or is an `in` expression (§8.7), the in-guard is active inside the then-branch only.

4 **Branch consistency**: if one branch consumes a linear variable, all branches shall consume it. If branches disagree on the linear state of a variable from the enclosing scope, the program is *ill-formed* (`[E003]`).

9.7 While statement

1 `while cond { body }` repeatedly evaluates *cond* and, if true, executes *body*.

> **Constraint**
>
> An unbounded `while` loop (one without a `decreasing` measure) shall not appear inside a `func`. Such use is *ill-formed* (`[E011]`).

> **Constraint**
>
> *cond* shall have type `bool`.

2 **In-guard**: when *cond* contains or is an `in` expression, the in-guard is active inside the loop body for each iteration.

9.7.1 Bounded while statement

3 `while cond decreasing M { body }` is a bounded loop. In addition to the while semantics above, the implementation shall statically verify:

1. **Non-negativity** (`[E080]`): the implementation shall prove that $M \geq 0$ whenever *cond* is true.
2. **Analyzable measure** (`[E081]`): M shall be in a form from which the termination checker can extract the variables it depends on. A measure the checker cannot analyze at all is rejected with `[E081]`. (A well-formed measure that merely fails to stay non-negative or to decrease — including a constant, or one over variables the body never assigns — is reported by `[E080]`.)
3. **Strict decrease** (`[E082]`): the implementation shall prove that every execution of *body* strictly decreases the value of M.

4 A bounded `while` loop is permitted inside a `func` because the implementation has statically verified termination.

5 The following patterns are accepted by the standard verification algorithm:

Condition	Measure	Why
`i < n`	`n - i`	i increases ⇒ measure decreases
`n > 0`	`n`	n decreases directly
`a < b`	`b - a`	either a increases or b decreases
`i in arr`	`arr.len - i`	`in` implies $i <$ `arr.len`

Example

```
func count(n int) int {
    var i = 0
    while i < n decreasing n - i {
        i = i + 1
    }
    return i
}
```

Rationale

The `decreasing` keyword bridges the gap between pure functions (which must terminate) and the common pattern of finite-input state machines (lexers, parsers) that are provably terminating. The programmer states the termination argument; the compiler verifies it.

9.8 Return statement

1 `return [expr]` terminates execution of the current function or procedure and transfers the value of *expr* (if present) to the caller.

Constraint

At the point of a `return`, all linear variables in scope shall have been consumed. A live (Unconsumed) linear variable at `return` is *ill-formed* (`[E003]`).

2 A `return` expression may be prefixed with `mov` or `var`: `return mov x` transfers ownership of `x` explicitly, and `return var x` returns a mutable borrow of `x` (the pattern used by a `func` whose return type is `var T`). Ownership transfer is also implicit from the function's declared return type, so a bare `return x` suffices when that type is owned.

Constraint

A `return var x` (or `return mov x`) that would escape a borrow of a *local* variable — one whose storage does not outlive the call — is a dangling reference and is *ill-formed* (`[E010]`). Returning a `var` borrow tied to a `var` parameter is permitted.

9.9 Defer statement

1 `defer { body }` schedules *body* for execution immediately before the enclosing scope exits, on all exit paths (`return`, `break`, `continue`, or fall-through). Multiple `defer` statements execute in LIFO order.

2 **Interaction with linear types**: if a linear variable is consumed inside a `defer` body, the variable is marked as defer-consumed. It is treated as unconsumed for the purposes of the main body (it may be used before the scope exit) and as consumed at scope exit.

Constraint

A `defer` body shall not contain `return`, `break`, or `continue` statements that would transfer control out of the deferred body.

Example

```
import std.fs

proc process() int {
    var f = open_file("data.txt", "r")
    defer { close_file(mov f) }   // f consumed at scope exit
    // ... use f ...
    return 0
}
```

9.10 Unsafe block

1 `unsafe { body }` is an unsafe block (see § 18). It creates a scope in which certain static checks are relaxed.

9.11 Comptime if statement

1 `comptime if cond { then } [else { else }]` evaluates *cond* at compile time. Only the taken branch is type-checked and emitted; the other branch is discarded. *cond* shall be a compile-time constant expression (§ 14).

Note

`comptime if` is the Lain equivalent of C preprocessor `#if`: it enables platform-specific or type-specific code selection without dead-code warnings.

9.12 For statement

1 `for i in start..end { body }` iterates with i ranging from *start* (inclusive) to *end* (exclusive). The loop variable i is immutable; its value range is statically known to be $[start, end - 1]$ throughout the body, enabling safe array indexing without runtime bounds checks.

2 `for` loops are permitted in both `func` and `proc`.

Constraint

start and *end* shall have integer types.

Chapter 10

Declarations

1 This clause specifies top-level declarations. Local variable declarations are specified in § 9.3. The grammar productions are in Annex A (*function-declaration*, *struct-declaration*, *enum-declaration*).

10.1 Declaration attributes

1 A declaration may be preceded by zero or more attributes. An attribute has the form **[name]** or **[name(args)]**, where *name* is a recognized attribute identifier from the closed set in Table 10.1, and *args* (if present) is a comma-separated list of expressions. Multiple attributes on a single declaration are written one per line.

```
[fast_math]
func dot(a f64[], b f64[], n int) f64 { ... }

[private]
func internal_helper() int { ... }

[fast_math]
[private]
func hot_path() f64 { ... }
```

2 Recognized attribute names (closed set, version 1.1):

Table 10.1: Recognized attribute names

Attribute	Applies to	Purpose
[fast_math]	`func`, `proc`	Enable FMA fusion (P4 opt-out, local).
[private]	any declaration	Module-private visibility.

Constraint

Q-017. An attribute name not listed in Table 10.1 is *ill-formed* (**[E103]**). An attribute that omits the identifier (e.g. **[ ]** or **[(arg)]**) is *ill-formed* (**[E102]**).

Note

Attributes are recognized at declaration position only. At expression position, **[** and **]** continue to delimit array and slice subscripts and array literals; the parser disambiguates by context.

10.2 Struct declarations

1 A struct declaration introduces a named product type with one or more named fields. Fields are listed in declaration order; order is significant for construction and layout.

```
type Point {
    x i32
    y i32
}
```

2 A field may be annotated with `mov`, making the field (and therefore the containing struct) a linear type:

```
type File {
    mov handle *FILE
}
```

> **Note**
>
> A struct with zero fields (a *unit* struct, `type Empty {}`) is permitted: it is a zero-sized marker type, constructed as `Empty()`.

10.3 Enum and ADT declarations

1 An enum declaration introduces a type whose values are drawn from a fixed, named set of variants. If every variant has no associated fields, the type is a simple enum (§7.6). If at least one variant has associated fields, the type is an algebraic data type (ADT) (§7.7).

```
type Direction {                // simple enum (one variant per
    line)
    North
    South
    East
    West
}

type Shape {                                    // ADT
    Circle     { radius i32 }
    Rectangle  { width i32, height i32 }
    Point
}
```

10.4 Function declarations

1 A function declaration introduces a `func`: a pure, terminating, side-effect-free callable. Its grammar production is *function-declaration*.

2 The signature consists of: the keyword `func`, a name, an optional list of type parameters (§14), a parameter list, and a return type annotation.

10.4.1 Parameters

3 Each parameter has a name, a type, and an ownership mode. The ownership mode determines how the argument is passed:

Syntax	Mode	ABI (struct T)
`name T`	shared (read-only)	`const T*`
`var name T`	mutable	`T*`
`mov name T`	owned (transfer)	`T` (by value)

Note

For primitive types (integer, float, bool), the ABI passes the value directly regardless of ownership mode, as for primitive C arguments.

4 A parameter whose declared type is the built-in `type` — written `name type`, with no `comptime` prefix — is a compile-time type parameter; it is bound to a concrete type at every call site, by inference from a value argument or explicitly (§ 14).

10.4.2 Return type annotation

5 The return type is written after the parameter list. It may carry a value range constraint:

```
func abs(x int) int >= 0 { ... }
```

Constraint

All return paths of a `func` shall produce a value whose type and value range satisfy the declared return type and constraint.

10.4.3 Purity constraints

Constraint

A `func` shall not contain an unbounded `while` loop (one without a `decreasing` measure) (`[E011]`).

Constraint

A `func` shall not call a `proc` or `extern proc` (`[E011]`).

Constraint

A `func` shall not call itself directly or indirectly (mutual recursion) (`[E011]`).

10.5 Procedure declarations

1 A procedure declaration introduces a `proc`: a callable with unrestricted side effects. Its syntax is identical to `func` except for the `proc` keyword. A `proc` may contain unbounded loops, call other `proc`s, and access global mutable state.

2 The entry point of every Lain program shall be declared as:

```
proc main() int { ... }
```

Constraint

A complete *program* shall define exactly one `proc main()int` as its entry point. An individual module (translation unit) compiled on its own — e.g. a library — need not

define `main`: the reference compiler accepts a `main`-less source file, and a program missing an entry point is diagnosed when the translation outputs are linked.

10.6 Extern declarations

1 An extern declaration introduces a C function into the Lain name space without providing a body. See §17 for full semantics.

```
extern proc printf(fmt *u8, ...) int
extern func strlen(s *u8) usize
```

Constraint

Variadic parameters `(...)` shall only appear in `extern` declarations. User-defined variadic functions are *ill-formed*.

10.7 Comptime declarations

Note

Top-level `comptime { ... }` blocks — evaluated by the compiler before any runtime code is emitted, to define compile-time constants or trigger `compileError` assertions — are a *planned* feature not yet accepted by the reference implementation (a top-level `comptime` block is presently `[E100]`). The `comptime` keyword is nonetheless reserved, and `comptime if` (§9.11) is available.

Chapter 11

Ownership and linearity

1 This clause specifies Lain's linear type system, ownership modes, borrow checker, and non-lexical lifetime rules. All rules in this clause are enforced at compile time with zero runtime overhead.

11.1 Ownership modes

1 Every parameter, return value, and struct field has one of three ownership modes:

Mode	Param syntax	Call-site	Semantics
Shared	`name T`	`f(x)`	read-only; multiple concurrent borrows allowed
Mutable	`var name T`	`f(var x)`	exclusive read-write; no other borrow concurrent
Owned	`mov name T`	`f(mov x)`	transfers ownership; source becomes Consumed

2 Local variables declared with `var name = expr` have mutable storage accessible within their scope; their ownership mode in the type system is Owned. When the value is of a *linear* type, an Owned local must be consumed before its scope exits (`[E003]`); a local of an unrestricted type carries no such obligation.

11.2 Linear states

1 Every variable of a linear type has a *linear state* at each program point. The three states are:

Uninit The variable has been declared but no value has been assigned.

Unconsumed The variable holds a value that has not yet been transferred.

Consumed The variable's value has been transferred via `mov` or a consuming function call.

2 The state machine governing transitions is:

State before	Operation	State after
Uninit	first assignment	Unconsumed
Unconsumed	`mov x` (transfer)	Consumed
Unconsumed	borrow (`var` or shared)	Unconsumed (borrow active)
Consumed	assignment (overwrite)	Unconsumed
Consumed	read or `mov`	*ill-formed*
Uninit	read or `mov`	*ill-formed*

Note

The Consumed → Unconsumed transition via overwrite enables the loop-reassign-then-consume pattern: a linear variable declared outside a loop may be consumed and reassigned on each iteration without leaking.

11.3 Diagnostics

Constraint

Using a variable, or a reference whose owner has been moved, after that value has been transferred (use-after-move; the Consumed state) is *ill-formed* (`[E001]`). Using a variable *before* it is initialized — the Uninit state — is a different diagnostic, `[E005]` (below).

Constraint

Consuming a struct field that has already been consumed (a double-consume at field granularity) is *ill-formed* (`[E002]`).

Constraint

A linear variable that is in the Unconsumed state when its scope exits (resource leak) is *ill-formed* (`[E003]`).

Constraint

An aliasing violation — attempting a mutable borrow when a shared borrow is active, or a shared borrow when a mutable borrow is active, or a second mutable borrow of the same variable — is *ill-formed* (`[E004]`).

Constraint

Reading a variable, struct field, or array element before it has been initialized (use of an uninitialized value; the Uninit state) is *ill-formed* (`[E005]`). Definite-assignment analysis tracks initialization per binding and per field.

Constraint

Consuming a linear variable or field inside a loop body that may execute more than once is *ill-formed* (`[E006]`).

Constraint

Passing a linear value to an owned (`mov`) parameter without the explicit `mov` keyword at the call site — an implicit move — is *ill-formed* (`[E007]`). (Consuming a linear value on some branches but not others is a separate diagnostic, `[E016]`, below.)

Constraint

Moving a value that is currently borrowed (`var` or shared), or moving a struct after some of its fields have already been consumed, is *ill-formed* (`[E008]`).

Constraint

If a linear value is in state CONSUMED on some control-flow paths through an `if`/`case` statement and in state UNCONSUMED on others, the merged state at the join point is undefined and the program is *ill-formed* (`[E016]`). The same diagnostic applies at field level: if a linear field of a struct is consumed on some paths but not on others, `[E016]` is issued for that field.

11.4 Mutable parameters for primitive types (`var T`)

A parameter declared `var name T`, where `T` is a primitive type (`iN`, `uN`, `f32`, `f64`, `bool`), is passed **by pointer** in the C ABI: the parameter's C type is `T*`.

Write-through semantics: an assignment to such a parameter inside the function body writes through the pointer and is visible to the caller. Reading the parameter's value automatically dereferences the pointer.

Example

```
proc increment(var n i32) {
    n = n + 1    // writes through: caller's variable is updated
}

proc main() i32 {
    var x = 0
    increment(var x)
    return x     // x == 1
}
```

Forwarding: when a `var T` primitive parameter is passed as argument to another `var T` parameter, the pointer is forwarded directly without dereferencing and re-taking the address. This behavior is identical to mutable parameters of struct type, where the by-pointer ABI was already in effect. The distinction is that for primitive types the codegen previously did not dereference reads; this was a bug that has been corrected.

Note

The call site shall annotate the argument with `var`: `f(var x)`. Omitting `var` passes a shared (read-only) borrow, which does not permit writes through the callee.

11.5 The read-write lock invariant

Invariant: At every program point, for every variable V, exactly one of the following holds:

1. V has $N \geq 0$ active shared borrows and zero active mutable borrows.
2. V has exactly one active mutable borrow and zero shared borrows.

This invariant is the compile-time analogue of a reader-writer lock. A conforming implementation shall enforce it at every program point.

11.6 Non-lexical lifetimes (NLL)

A borrow's lifetime ends at the borrow's *last use*, not at the end of the variable's lexical scope. The implementation computes last-use points via a use-analysis pre-pass over the AST.

Example

```
var data = Data(42)
var r = get_ref(var data)  // mutable borrow starts
use_ref(var r)             // last use of r: borrow ends HERE
    (NLL)
var y = read(data)         // OK: borrow has expired
```

11.7 Two-phase borrows

1 During argument evaluation for a function call, borrows of the receiver object are placed in the *RESERVED* phase. A RESERVED borrow allows concurrent shared reads of the same owner but blocks mutable writes and moves. After all arguments have been evaluated, RESERVED borrows are promoted to the *ACTIVE* phase.

2 This mechanism enables UFCS calls of the form `x.f(x.field)` where the first argument (the receiver) requires a mutable borrow of `x` while the second argument reads a field of `x`.

Phase	Allows on owner	Blocks on owner
RESERVED	shared reads	mutable writes, moves
ACTIVE	(nothing)	all other access

Example

```
proc push_n(var v Vec, n int) { v.data = v.data + n }

var v = Vec(0, 10)
v.push_n(v.cap)    // desugars to push_n(var v, v.cap)
                   // var v enters RESERVED phase; v.cap read is
                       allowed
```

11.8 Move semantics

1 The `mov` operator transfers ownership. The source variable transitions to Consumed state; any subsequent read or move of the source is *ill-formed* (`[E001]`).

2 Ownership transfer is the only way to pass a linear value to a consuming parameter. The `mov` keyword shall be present at the call site: `f(mov x)`.

11.9 Borrowed match

1 The `case &expr { ... }` form (see § 15) performs pattern matching without consuming the scrutinee. A temporary shared borrow of the scrutinee is registered for the duration of the match body and released when the match completes.

Constraint

Inside a `case &expr` body, the scrutinee shall not be mutated or moved. Attempting to do so violates the active shared borrow (`[E004]`).

11.10 Loop and ownership

A linear variable declared outside a loop may be consumed inside the loop body if and only if it is also reassigned (overwritten) on every path through the loop body before the next iteration begins.

Example

```
import std.fs

proc main() int {
    var f = open_file("data.txt", "r")
    close_file(mov f)          // consumed before loop
    var i = 0
    while i < 3 decreasing 3 - i {
        f = open_file("data.txt", "r")  // reassign (Consumed
            -> Unconsumed)
        close_file(mov f)               // consume
        i = i + 1
    }
    return 0
}
```

11.11 Branch consistency

After an `if`/`else` chain or `case` match, the linear state of each variable from the enclosing scope is the *intersection* of its states across all branches:

- If consumed in all branches: the variable is Consumed after the chain.
- If consumed in no branches: the variable is Unconsumed after the chain.
- If consumed in some but not all branches: *ill-formed* (`[E016]`).

The same rule applies at field level: if a linear field of a struct is consumed on some branches but not on others, `[E016]` is emitted for that field.

Example

```
type Handle { mov ptr *i32 }

func maybe_consume(mov h Handle, flag bool) {
    if flag {
        drop(mov h)    // consumed in then-branch
    }
    // [E016]: h consumed in then but not in else (implicit
        else)
}
```

Note

`[E007]` (generic ownership mismatch) was the diagnostic emitted before `[E016]` was introduced. `[E016]` is now the dedicated code for branch-inconsistency on linear values.

11.12 Field-level tracking

The borrow checker tracks borrows at the field level. Borrowing non-overlapping fields of the same struct simultaneously is permitted:

```
type Pair { a i32, b i32 }
var p = Pair(1, 2)
var ra = read_a(p.a)    // shared borrow of p.a (implicit)
var rb = read_b(p.b)    // shared borrow of p.b -- OK:
    non-overlapping
```

> **Note**
>
> The reference implementation tracks field borrows at one level of depth. Nested field borrows (e.g., `p.a.x` vs `p.a.y`) are conservatively treated as overlapping.

Chapter 12

Functions and procedures

12.1 The func/proc distinction

1 Lain distinguishes between pure functions (`func`) and side-effecting procedures (`proc`). This distinction is enforced statically and reflected in the type system.

Property	`func`	`proc`
Side effects	forbidden	allowed
Unbounded `while`	forbidden	allowed
Bounded `while decreasing`	allowed	allowed
`for` loops	allowed	allowed
Direct/indirect recursion	forbidden	allowed
Calling `proc`	forbidden	allowed
Calling `func`	allowed	allowed
Global mutable state	no access	full access
Termination guarantee	yes	no

2 A `func` is referentially transparent: calling it with the same arguments always produces the same result. Its evaluation has no observable effect on any state outside its own stack frame.

3 Local mutation through a `var T` parameter (exclusive mutable borrow) is permitted in a `func`. The borrow checker guarantees no aliasing, so the mutation is observable only through the parameter binding itself; the function remains deterministic and referentially transparent (same input borrow ⇒ same mutation pattern).

Example

```
type Counter { val i32 }

func increment(var c Counter) {  // legal: mutation is local
    c.val = c.val + 1
}
```

4 The compiler emits `[W130]` when a `proc` has no observable side effect — no calls to other `proc` or `extern proc`, no `panic`, no mutable global access, no unbounded `while`, no self-recursion — to suggest that it could be declared `func`. This is a non-blocking warning; the programmer may keep it as `proc` when future evolution will introduce effects.

12.2 Purity enforcement

> **Constraint**
>
> A `func` that calls a `proc` or `extern proc` is *ill-formed* (`[E011]`).

> **Constraint**
>
> A `func` that contains an unbounded `while` loop (one without a `decreasing` measure) is *ill-formed* (`[E011]`).

> **Constraint**
>
> A `func` that calls itself directly, or that participates in a mutual-recursion cycle, is *ill-formed* (`[E011]`). The implementation shall detect mutual recursion by performing a depth-first search on the call graph.

> **Note**
>
> Generic functions (`func` with `type` parameters) are excluded from mutual-recursion detection until they are monomorphized.

12.3 Termination analysis

1 A `func` is guaranteed to terminate because:

1. all `while` loops carry a verified termination measure;
2. `for` loops iterate over finite ranges;
3. recursion is forbidden.

2 No termination analysis is performed for `proc`s.

12.4 Universal Function Call Syntax (UFCS)

1 An expression of the form `x.f(args)` is desugared to `f(x, args)` before name resolution, provided `f` is not a declared field of `x`'s type. This is called Universal Function Call Syntax (UFCS).

2 **UFCS ownership**: the ownership mode of the first argument in the desugared call is determined by the declared mode of the first parameter of `f`:

- If the first parameter is `var`, the receiver is automatically mutably borrowed: `x.f()` desugars to `f(var x)`.
- If the first parameter is `mov`, the receiver is automatically moved: `x.f()` desugars to `f(mov x)`.
- If the first parameter is shared (default), the receiver is shared: `x.f()` desugars to `f(x)`.

3 **UFCS resolution algorithm**:

1. Check if `f` is a declared field of `x`'s type. If so, this is a field access, not UFCS.
2. Search for a function named `f` whose first parameter type is compatible with the type of `x`.
3. If found, desugar the call.

4. If no matching function is found, the call is unresolved. (The reference implementation does not yet emit a dedicated diagnostic for an unresolved UFCS call — it shares the undeclared-identifier gap noted in §6.3.)

Example

```
func is_empty(s u8[]) bool { return s.len == 0 }

var buf = "hello"
var empty = buf.is_empty()    // desugars to: is_empty(buf)
```

12.5 Calling conventions

The calling convention for Lain functions corresponds to the C99 calling convention of the target platform, as determined by the C compiler used for the translation output. The parameter passing rules are:

Lain parameter	C type	Semantics
shared struct `T`	`const T*`	pointer to read-only copy
`var` struct `T`	`T*`	pointer, caller's copy
`mov` struct `T`	`T`	by value (bitwise copy)
primitive any mode	`T`	by value

12.6 Return values

A return statement has the form `return <expr>`. The mode of the returned value is determined by the function's declared return type and follows the same rules as call-site argument passing.

A `return` may be prefixed with `mov` or `var`: `return mov x` makes the ownership transfer explicit, and `return var x` returns a mutable borrow (the pattern for a `func` whose declared return type is `var T`). When the return type is owned, a bare `return expr` already moves `expr`, so `mov` is optional there.

Note

A `return var x` / `return mov x` that would escape a borrow of a *local* variable is a dangling reference (`[E010]`, §9.8). This supersedes an earlier design (Q-009) in which explicit `mov`/`var` in return position was rejected.

12.7 The `panic` builtin

`panic(msg)` is a builtin expression that terminates program execution immediately. Its type is *Never* (the bottom type: a subtype of every type). Because it does not return, `panic` may appear in any expression position—including as the body of a match arm whose declared type is otherwise unrelated.

`panic` may be called from both `func` and `proc` contexts. The call is deterministic (same input yields the same termination), so it does not violate the purity of an enclosing `func`.

Constraint

Q-014, S16. When `panic` is executed, the program terminates immediately via the platform abort primitive. No `defer` block is executed. No stack unwinding occurs.

Note

The reference implementation emits, for `panic(msg)`, the equivalent of `fprintf(stderr, "panic: %.*s\n", ...) ; abort()`. The message is written to the standard error stream before abort.

Example

```
func head(s u8[]) u8 {
    if s.len == 0 {
        panic("empty slice")     // Never; satisfies the return type
    }
    return s[0]
}
```

12.8 Function attributes

A function declaration may be prefixed with one or more attributes (§ 10.1). The attributes applicable to functions in version 1.0 are:

- **`[fast_math]`** — enable fast floating-point optimizations (FMA fusion, contraction) for the body of this function. This is an opt-out from the determinism guarantee (P4) and applies only locally; functions without **`[fast_math]`** remain bit-deterministic within their target platform.
- **`[private]`** — module-private visibility (§ 16.5).

Note

The reference implementation translates **`[fast_math]`** on a function f to a pair of pragmas surrounding the body of f in the emitted C source, enabling `fp-contract=fast` for f only. The default (i.e., absence of **`[fast_math]`**) emits `-ffp-contract=off` flags, providing deterministic floating point within a single target platform and C compiler.

Chapter 13

Value range constraints

13.1 Overview

1 Value Range Analysis (VRA) is a static analysis that tracks integer value ranges through a program using interval arithmetic. It is used to:

- eliminate array bounds checks (§ 7.3);
- prove division-by-zero absence (§ 8.3);
- verify parameter and return constraints (§ 13.6);
- prove in-guard safety (§ 8.7).

2 VRA is decidable and polynomial in program size. No SMT solver is required. All analyses performed by a conforming implementation shall terminate.

13.2 Range representation

1 A value range is an interval $[\ell, u]$ where $\ell \leq u$. The sentinels $-\infty$ and $+\infty$ represent the absence of a lower or upper bound, respectively. A range may additionally be *unknown* (no information available), in which case the interval is $(-\infty, +\infty)$.

13.3 Range propagation rules

1 The following table defines the result range for binary arithmetic operations on intervals $[a, b]$ and $[c, d]$. All arithmetic is saturating: overflow wraps to $\pm\infty$.

Operation	Result range	Notes
$[a,b] + [c,d]$	$[a+c,\ b+d]$	saturating
$[a,b] - [c,d]$	$[a-d,\ b-c]$	saturating
$[a,b] \times [c,d]$	$[\min(ac,ad,bc,bd),\ \max(ac,ad,bc,bd)]$	saturating
$[a,b] \div [c,d]$, $c > 0$	$[a \div d,\ b \div c]$	integer truncation
$[a,b] \div [c,d]$, $0 \in [c,d]$	`[E015]`	division by zero
$[a,b]\ \%\ [c,d]$, $a \geq 0$, $c > 0$	$[0,\ d-1]$	
$[a,b]\ \%\ [c,d]$, otherwise	unknown	conservative

13.4 Conditional refinement

After an `if x < N` test, the VRA refines the range of `x` in the then-branch to $[\ell, N-1]$ and in the else-branch to $[N, u]$ (where $[\ell, u]$ is the range before the test).

At the join point after an `if`/`else`, the range is the union of the ranges from both branches.

When conditions are chained with `and`, refinements from the left operand are active when evaluating the right operand (§ 8.7).

13.5 In-guard semantics

An in-guard `idx in arr` (§ 8.7) introduces the constraint $0 \leq idx < arr.len$ into the VRA's range environment. Inside the guarded body, `arr[idx]` is accepted without further proof.

When `idx in arr` is the condition of a `while decreasing M` loop, the VRA infers that $arr.len - idx \geq 0$ (a valid non-negativity proof for the measure $arr.len - idx$).

13.6 Constraint annotations

A constraint annotation of the form `T op bound` on a parameter or return type declares an invariant that the VRA shall enforce:

- **Parameter constraint**: at every call site, the VRA shall prove that the argument satisfies the constraint.
- **Return constraint**: at every `return` statement, the VRA shall prove that the return expression satisfies the constraint.

> **Constraint**
>
> A function call where the argument range cannot be proven to satisfy a parameter constraint is *ill-formed* (`[E012]`).

> **Constraint**
>
> A `return` expression whose range cannot be proven to satisfy the declared return constraint is *ill-formed* (`[E086]`) — the same boundary mechanism (§ 13.10) that checks assignments and narrowing conversions. (A *parameter*-constraint violation at a call site is the distinct code `[E012]`, above.)

13.7 Interprocedural range propagation

When a call expression appears in a context where the VRA needs a range, it uses the callee's declared return constraint (if any) as a conservative approximation.

> **Example**

```
func nonzero() int >= 1 { return 42 }

func divide(a int) int {
    return a / nonzero()    // divisor range known >= 1: no [E015]
}
```

When the callee has no declared return constraint, the call expression has an unknown range.

Note

Interprocedural VRA is conservative: only declared constraints are used, not body analysis. This preserves decidability and ensures that changing a function's body cannot silently affect its callers' safety proofs.

13.8 Limits and known gaps

1 The following limitations are known and accepted:

1. VRA does not track general integer value ranges of struct fields: a field's value range is unknown (a field read through a pointer always is). Two narrower field properties *are* tracked — per-field *initialization* (definite assignment, § 11.3) and a struct field's *slice length* within an `and`-scope (to prove indexing of a field slice).
2. VRA does not track ranges across loop-carried dependences with non-linear measures.
3. Integer overflow in VRA arithmetic uses saturating semantics; the actual Lain program uses wrapping arithmetic. Programs whose correctness depends on integer overflow are not analyzed correctly by VRA.
4. Floating-point values are not tracked by VRA.

13.9 Loop body affine recap

1 For `for V in start..end` loops whose body contains assignments of the form `x = x + c` or `x = x - c` with constant step c, the implementation computes the post-loop range of x precisely:

$$x_{\text{post}} \in [x_{\text{init}} + c \cdot n_{\min},\ x_{\text{init}} + c \cdot n_{\max}]$$

where $n_{\min}$, $n_{\max}$ is the iteration count derived from the loop's range. This avoids the conservative widening that would otherwise mark x as unknown after the loop.

Example

```
var x = 0
for i in 0..10 {
    x = x + 1
}
// Post-loop: x is exactly 10 (range [10, 10]).
```

2 Patterns not matching the affine form (mixed updates in branches, non-constant step, multiple variables updated together) fall back to the conservative widening (unknown post-loop range).

13.10 Overflow-at-boundary check

1 The Phase 5 boundary check applies at every integer-type boundary where a value crosses into a narrower or different-domain target. The check sites are:

- Variable declaration (`var x T = expr`).
- Assignment to identifier, field, or indexed element (`x = expr`, `p.f = expr`, `arr[i] = expr`).

- Return value vs the function's declared return type.
- Call argument vs the parameter's declared type.
- Struct constructor field initialization.

2 At each site, if the source value's VRA range exceeds the target type's natural range, the program is *ill-formed* (`[E086]`). The diagnostic offers four resolutions: widen the target, use a wrapping operator, use a saturating operator, or tighten input constraints so VRA can prove safety.

3 The check is skipped:

- Inside an `unsafe` block;
- When the source range is unbounded (the analysis cannot rule out overflow without further information; the user must annotate);
- For integer-literal sources (literals are polymorphic across `iN`/`uN` and trusted to fit when written explicitly).

4 The wrapping/saturating operators (`+%`, `*%`, `+|`, `*|`) produce a result whose range is automatically clamped to the LHS type's natural range, so they pass the boundary check unconditionally.

Example

```
var a u8 = 200
var b u8 = 100
// var bad u8 = a + b       // [E086]: range [300, 300] exceeds
    u8
var w u8 = a +% b            // OK: wrapping clamps to [0, 255]
var s u8 = a +| b            // OK: saturating clamps to [0, 255]
```

13.11 Sub-slice bounds

1 A sub-slice expression `arr[a..b]` produces a slice covering the half-open range $[a, b)$ of `arr`. The inferred type of the result is `T[b-a]`, a sized slice (§ 7.4.1) whose length is statically known. When both endpoints are compile-time integer literals, the compiler verifies $a \leq b$; if the check fails the program is *ill-formed* (`[E087]`).

2 When the endpoints are variables, each is range-checked against the container length (`[E085]`, the standard bounds check). A runtime guarantee on the ordering of variable endpoints is not emitted at compile time; the programmer must establish it through refinement constraints or by limiting the range expression to literal bounds.

Example

```
var arr i32[10] = [0,1,2,3,4,5,6,7,8,9]
var ok  = arr[2..5]     // type i32[3], half-open, length 3
// var bad = arr[5..2] // [E087]: start 5 > end 2
var empty = arr[3..3]   // OK: empty slice of length 0
```

13.12 Sized slice call-site verification

1 When a function parameter is declared with a sized slice type (§ 7.4.1), the compiler verifies at every call site that the supplied argument satisfies the parameter's length constraint. A

proved violation is *ill-formed* (`[E087]`). The check is conservative: it fires only when VRA can *prove* the constraint is not met; it is silent when the constraint cannot be determined statically.

2 The cases checked are:

- `T[>= k]` or `T[> k]` with integer literal `k`: fires if the argument length's VRA upper bound is less than the required minimum (`>= k` requires $len \geq k$; `> k` requires $len \geq k + 1$).
- `T[k]` (exact length, literal `k`): fires if the argument length's VRA range does not contain `k`.
- `T[ref.len]` (cross-parameter equality): fires if both sides have a concrete singleton VRA range and they differ.

Example

```
func pair_sum(arr i32[>= 2]) i32 { return arr[0] + arr[1] }

proc main() i32 {
    var one i32[1] = [42]
    // pair_sum(one)        // [E087]: len 1 < required 2
    var two i32[2] = [1, 2]
    return pair_sum(two)    // OK: len 2 satisfies >= 2
}
```

3 **Sized slice VRA entry.** At function entry, for each parameter `name T[expr]` the compiler seeds the range table with the constraint derived from the size expression:

- `T[k]` (literal): `__len_name` $\in [k, k]$.
- `T[>= k]` (literal): `__len_name` $\in [k, +\infty)$.
- `T[> k]` (literal): `__len_name` $\in [k{+}1, +\infty)$.
- `T[a.len]` (member reference): difference constraint `__len_name` $=$ `__len_a`.
- `T[a.len + b.len]` (arithmetic): constraints `__len_a` $\leq$ `__len_name` and `__len_b` $\leq$ `__len_name`.

These constraints are available to the VRA throughout the function body, enabling the Omega Test (§13.14) to discharge complex indexing patterns.

13.13 VRA analysis levels

1 Lain's VRA applies four escalating analysis levels at each bounds-check site. The levels are tried in order; if one succeeds (the check is proved safe), the remaining levels are skipped.

L1 — Interval arithmetic. Each variable is represented by an interval $[\ell, u]$. The check passes if the interval of the index falls within $[0, arr.len - 1]$ directly.

L2 — Difference constraints. A system of difference constraints of the form $x - y \leq c$ is maintained. The check passes if the constraint system can derive the required bound in one step.

L3 — One-step bridge (transitivity). Two-hop transitivity: if $v_1 - X \leq a$ and $X - v_2 \leq b$ are both known, the checker derives $v_1 - v_2 \leq a + b$ without running the full Fourier-Motzkin algorithm. Handles patterns such as `i < src.len - 1` via the chain $i < src.len$ and $src.len - 1 < src.len$.

L4 — Fourier-Motzkin linear arithmetic (Omega Test). Full linear arithmetic elimination over all VRA variables. See §13.14.

All four levels are polynomial time. No SMT solver is used.

13.14 Omega Test — Fourier-Motzkin prover (VRA level 4)

When L1–L3 fail to discharge a bounds check, the implementation invokes the *Omega Test*: a self-contained Fourier-Motzkin linear arithmetic prover (roughly 410 lines in `src/sema/omega.h`, no external dependencies).

Proof method (proof by contradiction). To prove $index < bound$:

1. Decompose both expressions into linear forms $\sum_i c_i \cdot x_i + k$ over named variables. References to `x.len` are mapped to the synthetic VRA variable `__len_x`.
2. Form the negated query: $bound - index \leq 0$.
3. Extract all VRA range and difference constraints that mention the variables appearing in the query.
4. Run Fourier-Motzkin variable elimination. Each step crosses upper-bound inequalities with lower-bound inequalities to produce a system with one fewer variable.
5. If the resulting constant system contains an inequality $0 \leq c$ with $c < 0$, the system is UNSAT: the negation is impossible, so $index < bound$ always holds.

The prover also supports non-negativity queries: to prove $expr \geq 0$, it shows that $expr \leq -1$, combined with VRA constraints, is UNSAT.

Example patterns proved by the Omega Test.

Example

```
// concat: out[j + a.len] with j < b.len and out.len == a.len + b.len
// FM eliminates j; the cross-pair (j < b.len, j >= b.len) gives 0 <= -1 (UNSAT).
func concat(a i32[], b i32[], out i32[a.len + b.len]) {
    for j in 0..b.len {
        out[j + a.len] = b[j]    // proved safe by Omega
    }
}

// reverse: out[n - i - 1] with n = arr.len and i < n
// FM eliminates n and __len_arr; reduces to (i <= -1) AND (i >= 0) (UNSAT).
func reverse(arr i32[], out i32[arr.len]) {
    var n = arr.len
    for i in 0..n {
        out[n - i - 1] = arr[i]    // proved safe by Omega
    }
```

```
}
```

Equality propagation for local length aliases. When the body contains `var n = x.len`, the compiler registers the equality $n - \texttt{\_\_len\_x} = 0$ (expressed as two difference constraints) so the Omega Test can cancel `n` and `__len_x` in the same linear form.

Conservatism. If the FM buffer overflows (more than 80 inequalities in the working system), the prover gives up and returns *unknown*; the check then falls through to E085 as if L4 had not fired. This bound is unreachable for the small systems (2–6 variables, 5–15 inequalities) typical of slice manipulation code.

Chapter 14

Generics and compile-time evaluation

14.1 Overview

1 Lain implements genericity through compile-time function evaluation (CTFE). Generic types and functions accept *type parameters* (declared as `name type`) or *compile-time value parameters* (declared as `comptime name non-type`). The compiler evaluates them at compile time and monomorphizes the result. There is no type erasure and no runtime polymorphism.

> **Note**
>
> **Implementation status.** The reference compiler implements the *core* of this chapter and monomorphizes it: generic `func`s with `type` parameters (bound by inference from a value argument or by an explicit type argument), and generic types written `type Name(T type)= type { ... }`. The remaining constructs below are *planned* and not yet accepted — compile-time *value* parameters / const generics (`comptime N int`), `comptime { ... }` expression blocks, and the `func`-returning-`type` spelling; each is flagged at its section, and is presently reported as `[E100]`. (`comptime if`, § 14.7, *is* available.)

14.2 Type parameters

1 A parameter declared `name type` is a type parameter. Types are implicitly compile-time — the `comptime` keyword shall *not* precede a type parameter. At every call site, the argument shall be a type expression known at compile time.

```
func identity(T type, x T) T {
    return x
}

var n = identity(int, 42)        // T = int, monomorphized
var s = identity(u8[:0], "hi")   // T = u8[:0], separate
    monomorphization
```

2 Type parameters are evaluated entirely at compile time; they produce no runtime code.

> **Constraint**
>
> The `comptime` keyword shall not appear before a parameter whose type is `type`. `func f(comptime T type)` is ill-formed; write `func f(T type)` instead.

14.3 Compile-time value parameters

1 A parameter declared `comptime name non-type` is a compile-time value parameter. The argument shall be a constant expression of the declared type known at compile time.

> **Note**
>
> **Planned (not yet implemented).** Compile-time value parameters (const generics) are not accepted by the reference compiler: a declaration such as the one below is presently `[E100]`.

```
type RingBuffer(T type, comptime N int) = type {
    items T[N]
    head  usize
    tail  usize
}
```

> **Constraint**
>
> A compile-time evaluation context (CTFE) shall not call a `proc` or an `extern proc`; only `func` calls are admissible. A violation is *ill-formed* (`[E101]`).

14.4 Type as value

1 A `func` with return type `type` is a type constructor. Its body shall consist only of expressions and statements that are evaluable at compile time. The return value is a type that the caller may use in type annotations and variable declarations.

```
type Option(T type) = type {
    Some { value T }
    None
}

var x = Option(int).Some(42)
```

2 Type constructors may also be written as `func` declarations returning `type`:

> **Note**
>
> **Planned (not yet implemented).** The `func`-returning-`type` spelling below is presently `[E100]`; use the `type Name(T type)= type { ... }` form above, which is accepted.

```
func Option(T type) type {
    return type {
        Some { value T }
        None
    }
}
```

> **Constraint**
>
> A function returning `type` shall be declared as `func` (not `proc`).

14.5 Monomorphization

1 For each unique combination of comptime arguments at call sites, the implementation produces a separate instantiation of the function. The name of each instantiation in the translation output is *implementation-defined*[1].

2 Monomorphization is a compile-time-only operation; at runtime, each call site calls a specific, non-generic function.

14.6 Comptime expression blocks

1 A `comptime { expr }` expression evaluates *expr* at compile time. The result is a compile-time constant that may be used as a type annotation, array size, or parameter constraint value.

> **Note**
>
> **Planned (not yet implemented).** `comptime { ... }` expression blocks are not accepted by the reference compiler (presently `[E100]`). The statement form `comptime if` (§14.7) *is* available.

> **Constraint**
>
> A `comptime` expression shall be pure and terminating (i.e., it may only use `func` calls and the constructs permitted in `func` bodies).

14.7 Comptime if

1 `comptime if cond { ... } else { ... }` is a compile-time conditional. Only the taken branch is type-checked and emitted; the other branch is entirely discarded. *cond* shall evaluate to a compile-time boolean constant.

> **Note**
>
> `comptime if` enables platform-specific code selection without dead-code warnings. It is the preferred way to write code that varies based on compile-time parameters.

14.8 Comptime recursion depth

1 The planned compile-time evaluation engine (§14.3, §14.6) will bound the recursion depth of comptime function evaluation *implementation-defined*[2] and reject an evaluation that exceeds it. Since that engine is not yet implemented, no depth diagnostic is currently emitted.

14.9 Mode preservation in generics

1 When a type parameter `T` is substituted with a concrete type, the ownership mode of the parameter is preserved. If a parameter was declared `var T`, and `T` is substituted with a type that has mode `shared` in its declaration, the substituted parameter retains the `var` mode.

> **Note**
>
> This rule ensures that mode annotations in generic functions behave consistently regardless of the concrete type argument.

[1] Implementation-defined behavior: determined by the implementation

[2] Implementation-defined behavior: by an implementation-defined limit

Chapter 15

Pattern matching

15.1 Overview

The `case` expression matches a value against a sequence of patterns and evaluates the body of the first matching arm. Every `case` expression shall be exhaustive: the patterns shall collectively cover all possible values of the scrutinee type. The grammar production is *match-expression* (Annex A).

15.2 Syntax

```
case expr {
    Pattern1: body1
    Pattern2: body2
    else:     default_body
}
```

Each arm consists of a pattern (or comma-separated list of patterns) and a body. The body may be a single expression or a block. The arms are tested in declaration order; the first matching arm is taken.

15.3 Patterns

The following pattern forms are supported:

Wildcard (`else`) Matches any value. Must be the last arm. Required for integer and character scrutinees.

Literal pattern Matches a specific integer or character value: `42:`, `'a':`.

Enum/ADT variant pattern Matches a specific variant. If the variant has fields, they are bound: `Circle(r): ...` binds the radius to `r`.

Range pattern Matches a range of integer or character values. Consistent with expression ranges and sub-slices (§5.8, §8.13), `a..b` is *exclusive* of the upper bound and `a..=b` is *inclusive*: `1..10:` matches 1 through 9, and `1..=10:` matches 1 through 10.

Multi-pattern (OR) A comma-separated list of patterns on a single arm: `1, 2, 3: ...`.

15.4 Exhaustiveness

A `case` expression is exhaustive if the set of patterns covers all values of the scrutinee type. The rules by type are:

Scrutinee type	Exhaustiveness requirement
Enum	All variants covered, or `else` arm present
ADT	All variants covered, or `else` arm present
Integer	`else` arm always required
`u8` (char)	`else` arm always required
`bool`	`true` and `false` covered, or `else`

Constraint

A non-exhaustive `case` expression is *ill-formed* (`[E014]`).

Example

```
type Color {
    Red
    Green
    Blue
}
var c = Color.Red

case c {
    Red:   return 1
    Green: return 2
    Blue:  return 3
}  // exhaustive: all three variants covered
```

15.5 Ownership and match

By default, `case expr` may consume the scrutinee if it is a linear type. Bindings introduced by variant patterns in consuming arms hold ownership of the bound values.

`case &expr` performs a non-consuming match: the scrutinee is not consumed but instead temporarily borrowed for the duration of the match body (see § 11.9).

Constraint

Inside a `case &expr` body, the scrutinee shall not be mutated or moved (`[E004]`).

15.6 Case expression result type

When `case` is used as an expression, all arms shall produce values of compatible types (§ 7.10). The type of the `case` expression is the common type.

Constraint

Arms of a `case` expression that produce incompatible types are *ill-formed*.

15.7 String pattern matching

String literals may appear as patterns. A string pattern matches if the scrutinee has the same byte sequence and length as the pattern literal. The comparison is performed byte-by-byte (analogous to `memcmp`).

Note

String pattern matching in the translation output is implemented as a sequence of byte comparisons or `memcmp` calls, depending on the implementation. Its asymptotic cost is O(n) in the pattern length.

Chapter 16

Module system

16.1 Overview

1 Each Lain source file defines one module. Modules are the unit of code organization and compilation. All names declared at the top level of a source file are in that module's scope and are accessible from other modules that import the module.

16.2 Module identity

1 A module's identity is *implementation-defined*[1]; there is *no* `module` declaration keyword (a leading `module ...` line is `[E100]`). The reference implementation names a module by its path relative to the working directory, with directory separators mapped to dots — so `std/io.ln` is the module `std.io`.

16.3 Import declarations

1 `import module.path` makes the declarations of the named module available in the current source file. The grammar production is *import-declaration* (Annex A).

2 **Selective import.** `import module.path.{name1, name2, ...}` brings only the listed names into scope rather than the whole module. This is the form used throughout the standard library, e.g. `import std.mem.{mem_alloc, mem_free}` or `import std.fs.{File, open_file, close_file}`.

3 The `use` statement (§ 5.5) provides a *local-scope* import/alias inside a function body. *implementation-defined*[2]; the standard library uses top-level `import` rather than `use`.

4 **Path resolution**: a module path is converted to a file system path by replacing dots with directory separators and appending `.ln`. The search is relative to the compiler's working directory. *implementation-defined*[3].

Module path	File path
`std.c`	`std/c.ln`
`std.io`	`std/io.ln`
`std.fs`	`std/fs.ln`
`std.math`	`std/math.ln`
`mylib.utils`	`mylib/utils.ln`

[1] Implementation-defined behavior: derived from its file path by the implementation

[2] Implementation-defined behavior: Its precise semantics are implementation-defined in this revision

[3] Implementation-defined behavior: The exact search algorithm, including support for search paths, is implementation-defined

16.4 Module loading

1 When the compiler encounters an `import` declaration:

1. The module path is converted to a file path.
2. If the module has already been loaded, the import is a no-op (deduplication).
3. Otherwise, the file is read, lexed, parsed, and recursively resolved.
4. The loaded module's declarations are merged into the importing module's declaration list.

2 Each module is loaded at most once. Circular imports are therefore implicitly handled: if module A imports module B which imports module A, the second import of A is a no-op because A is already loaded.

> **Note**
>
> The reference implementation maintains a linked list of loaded modules keyed by their dot-separated name.

16.5 Name visibility

1 By default, every name declared at module scope is *public*: visible to any module that imports the declaring module. This default matches the typical usage where module declarations form an API surface.

2 A declaration prefixed with the attribute `[private]` (§10) is *module-private*: it is visible only within the defining module and may not be referenced from any importing module.

```
// In module foo.bar:
func public_api() int { ... }     // public by default

[private]
func internal_helper() int { ... }    // only foo.bar may
    reference this
```

> **Constraint**
>
> **Q-018.** A reference to a `[private]` declaration from a module other than the one where it is defined is *ill-formed* (`[E084]`). The reference may appear in any expression position—identifier lookup, member access, type instantiation, etc.

> **Note**
>
> Compilation under the reference implementation translates each `[private]` declaration to a C declaration with internal linkage (`static` storage class), and each public declaration to one with external linkage.

16.6 Standard library module paths

1 The following module paths are defined by this specification for the standard library:

Module path	Description
`std.c`	C standard library bindings

Module path	Description
`std.io`	Console I/O
`std.fs`	File system operations
`std.math`	Mathematical functions

16.7 Name mangling

1 The name of a Lain declaration in the translation output is *implementation-defined*[4]. The reference implementation uses the function name prefixed with the module path, with dots replaced by underscores. Name collisions between modules are *implementation-defined*[5].

[4] Implementation-defined behavior: determined by the implementation

[5] Implementation-defined behavior: detected by the implementation; the reference implementation appends a numeric suffix to resolve conflicts

Chapter 17

C interoperability

17.1 Overview

1 Lain compiles to C99 and provides direct, zero-overhead interoperability with C libraries through three mechanisms:

1. `c_include` — injects C `#include` directives;
2. `extern` declarations — declares C functions for use in Lain;
3. type mapping — Lain types map to their C99 counterparts.

17.2 The `c_include` declaration

1 `c_include "header.h"` causes the compiler to emit a `#include` directive in the generated C code. The string content is emitted verbatim.

```
c_include "<stdio.h>"
c_include "<stdlib.h>"
```

> **Constraint**
>
> `c_include` shall appear only at module top level.

17.3 Extern function declarations

1 `extern func name(params)return` declares a C function that Lain treats as pure. It may be called from both `func` and `proc` contexts.

2 `extern proc name(params)return` declares a C function with potential side effects. It may be called only from `proc` contexts.

> **Constraint**
>
> Calling an `extern proc` from a `func` is *ill-formed* (`[E011]`).

> **Note**
>
> **Trust boundary — I-016.** An `extern func` declaration is a *programmer assertion* that the referenced C function is pure (deterministic, no side effects, no access to mutable global state). The Lain compiler cannot verify this property and trusts the declaration. Misuse—for example, declaring `libc_rand` or `malloc` as `extern func`—is a programmer error, not a compile error, and may invalidate the guarantees that depend on purity (notably P4

> determinism for any `func` that transitively calls the misdeclared extern). When in doubt, use `extern proc`.

3 Ownership annotations on extern parameters work identically to those on native Lain functions; the borrow checker tracks ownership of resources obtained via extern calls.

17.4 Type mapping

1 The following table defines the normative mapping between Lain types and their C99 equivalents. The mapping is the ABI used by the translation output.

Lain type	C99 type	Notes
`u8`	`uint8_t`	
`u16`	`uint16_t`	
`u32`	`uint32_t`	
`u64`	`uint64_t`	
`usize`	`size_t`	pointer-sized
`i8`	`int8_t`	
`i16`	`int16_t`	
`i32`	`int32_t`	
`i64`	`int64_t`	
`isize`	`ptrdiff_t`	pointer-sized
`int`	`int`	at least 32-bit
`f32`	`float`	IEEE 754
`f64`	`double`	IEEE 754
`bool`	`_Bool`	C99 bool (1 byte); *implementation-defined*[1]
shared struct `T`	`const T*`	
`var` struct `T`	`T*`	
`mov` struct `T`	`T`	by value
`T[]`	`struct { size_t len; T* data; }`	fat slice
`*T`	`const T*`	unsafe only
`var *T`	`T*`	unsafe only

17.5 ABI for function parameters

1 This table is normative and applies to all Lain function calls, including those to `extern` functions.

Lain param	Type	C param	Direction
`name T`	struct	`const T*`	in
`var name T`	struct	`T*`	in/out
`mov name T`	struct	`T`	in (by value)
any mode	primitive	`T`	in

[1] Implementation-defined behavior: 0 or 1

17.6 Pointer operations

1 Raw pointer operations (`*ptr` dereference, `&x` address-of) require an `unsafe` block (see § 18).

> **Constraint**
>
> Pointer-to-pointer or integer-to-pointer casts shall occur only inside an `unsafe` block (`[E012]`).

17.7 String interop

1 Lain strings (`u8[]`) are byte slices, not null-terminated C strings. To pass a Lain string to a C function expecting `char*`, the `.data` pointer must be extracted explicitly:

```
c_include "<stdio.h>"
extern proc puts(s *u8) int

var s = "hello"
puts(s.data)    // OK: .data is *u8 = const char*
```

2 String literals are null-terminated (§ 5.7.5), so `literal.data` is a valid `const char*` for C functions that expect a C string.

17.8 Opaque C types

1 An opaque C type is declared with `extern type Name`. The type's layout is unknown to Lain; it may only be used behind a pointer.

```
extern type FILE
extern proc fopen(path *u8, mode *u8) mov *FILE
```

> **Constraint**
>
> An opaque type shall not be used as a value type (only as a pointer target). `var f FILE` is *ill-formed*.

Chapter 18

Unsafe code

18.1 Purpose

1 The `unsafe` block provides a controlled escape hatch for operations that a conforming implementation cannot statically verify. Unsafe blocks are explicitly delimited in the source text and are therefore auditable. Unsafe code cannot accidentally spread outside the enclosing block.

18.2 Syntax

1 An unsafe block is written as `unsafe { body }`. It may appear wherever a statement is valid (§ 9.10).

18.3 What unsafe enables

1 Inside an `unsafe` block, the following operations are permitted:

1. **Raw pointer dereference**: `*ptr` reads the value pointed to by a raw pointer; `*ptr = v` writes through a mutable raw pointer.
2. **Address-of**: `&x` creates a raw pointer to the storage of `x`.
3. **Pointer and integer casts**: `p as *T`, `n as *T`, `p as usize`, etc.
4. **Direct ADT field access**: reading a variant's field with the qualified form `value.Variant.field` without going through a `case` arm. Outside `unsafe` this is *ill-formed* (`[E125]`); the safe idiom is to destructure with `case`.
5. **Bounds-check suppression**: array indexing inside an `unsafe` block does not require a VRA proof (the programmer asserts the access is in bounds).

18.4 What unsafe does NOT disable

1 The following safety features remain active inside `unsafe` blocks:

Feature	Disabled by `unsafe`?
Pointer dereference (`[E060]`)	**Yes**
Pointer/integer casts (`[E012]`)	**Yes**
Direct ADT field access (`[E125]`)	**Yes**
Bounds check requirement (`[E085]`)	**Yes**

Feature	Disabled by `unsafe`?
Ownership tracking (`[E001]`–`[E007]`)	No
Borrow checking (`[E004]`)	No
Move semantics	No
Type checking	No
Division-by-zero check (`[E015]`)	No
Exhaustiveness checking (`[E014]`)	No
Purity enforcement (`[E011]`)	No

Constraint

Ownership and linearity rules (§ 11) are not suspended inside `unsafe` blocks. Use-after-move, double-move, and borrow conflicts are still constraint violations inside `unsafe` (`[E001]`–`[E007]`).

18.5 Undefined behavior in unsafe

1 The following operations inside an `unsafe` block may produce **undefined behavior**[1]:

- Dereferencing a null pointer.
- Dereferencing a pointer to freed or uninitialized memory.
- Writing to memory through a pointer derived from a read-only (shared) source.
- Accessing an array element with an out-of-range index.
- Accessing a field of an ADT through a tag that does not match the active variant.

2 A conforming implementation that generates C99 translation output inherits the undefined behavior rules of ISO/IEC 9899 (C17) for operations within `unsafe` blocks.

18.6 Borrow scope for unsafe blocks

1 An `unsafe` block is a regular block scope for the purposes of borrow checking. Borrows registered within the block are released on block exit, and the borrow checker's invariants hold at the block boundaries.

18.7 Nesting

1 `unsafe` blocks may be nested. The inner block does not extend the permissions of the outer block beyond what is already enabled. *implementation-defined*[2].

[1] Undefined behavior: undefined behavior if the programmer's safety assertion is incorrect

[2] Implementation-defined behavior: Whether the implementation issues a diagnostic for redundant nested unsafe blocks is implementation-defined

Chapter 19

Memory model

19.1 Overview

1 Lain's memory model is founded on three invariants:

1. **Deterministic allocation** — every allocation is explicit and predictable; the implementation never implicitly allocates heap memory.
2. **Zero runtime overhead** — there is no garbage collector and no reference-counting mechanism.
3. **C99 layout compatibility** — the physical layout of all objects follows ISO/IEC 9899 (C17) rules exactly.

2 This clause specifies the abstract notions of *object*, *storage duration*, *object representation*, and *address*. The abstract machine described here is realised by any conforming translation output (§ 4).

19.2 Objects

1 An *object* is a contiguous region of storage that holds a value of a declared type. Every variable declaration introduces exactly one object. Fields of a struct introduce sub-objects. Elements of a fixed-size array introduce element sub-objects.

2 Every object has:

- a *declared type* (§ 7);
- an *ownership mode* — shared, `var`, or `mov` (§ 11.1);
- a *linear state* — Uninit, Unconsumed, or Consumed (§ 11.2);
- a *storage duration* (§ 19.3);
- an *address* — the byte address of its first storage unit, aligned as required by the target platform.

19.3 Storage duration

1 Three storage durations are defined:

Automatic duration Objects declared inside a function or block body (including function parameters) have automatic storage duration. Storage is allocated when the enclosing scope is entered and released when that scope exits.

Static duration Objects declared at module scope have static storage duration. Their storage persists for the entire program execution. *implementation-defined*[1].

Heap (explicit) duration Objects allocated through C interop (`malloc` or equivalent) have programmer-controlled duration. The ownership system (§ 11) enforces that `mov` pointers to heap memory are consumed exactly once (`[E003]`).

> **Constraint**
>
> A `return` statement shall not return a reference whose referent has automatic storage duration (`[E010]`).

19.4 Stack allocation

1 All local variables, struct values, and fixed-size arrays are allocated on the call stack. No implicit heap allocation is ever performed by the compiler.

```
var x int = 42      // stack: sizeof(int32_t) bytes
var arr int[100]     // stack: 100 * sizeof(int32_t) bytes
var p Point          // stack: sizeof(Point) bytes
```

2 Each function call produces a *stack frame* containing:

- all local variables declared in the function;
- all function parameters (by value or by pointer, per the calling convention of § 12.1);
- compiler-generated temporaries required for expression evaluation.

3 The stack frame is automatically reclaimed when the function returns. *implementation-defined*[2].

19.5 Heap allocation

1 Heap allocation is performed exclusively through C interop (§ 17). The idiomatic pattern declares the C `malloc`/`free` pair as `extern proc`:

```
c_include "<stdlib.h>"
extern proc malloc(size usize) mov *void
extern proc free(ptr mov *void)
```

2 Because `malloc` returns `mov *void`, the caller receives sole ownership of the allocated block. Because `free` consumes `mov *void`, calling `free` transfers that ownership to the allocator. The ownership system therefore enforces statically:

- **No use-after-free**: using the pointer after `free` is a use-after-move (`[E001]`).
- **No double-free**: a second call to `free` on the same pointer is a double-consume (`[E002]`).
- **No leak**: failing to call `free` on a `mov` pointer is an unconsumed linear value (`[E003]`).

[1] Implementation-defined behavior: The value of a static-duration object whose declaration lacks an initializer is implementation-defined; the reference implementation zero-initializes such objects via C99 static initialization rules

[2] Implementation-defined behavior: The implementation's behaviour when the call stack is exhausted is implementation-defined; the reference implementation inherits the platform's stack overflow signal

19.6 Value semantics

1 The behaviour of assignment and function argument passing is determined by the type's linearity and the ownership mode:

Type	Assignment `b = a`	Notes
Primitives (`int`, `bool`, ...)	Bitwise copy	Always allowed
Non-linear struct	Field-by-field copy	Allowed
Linear struct (has `mov` field)	Forbidden	Requires `mov`
Non-linear array	Element-wise copy	Allowed
Linear array	Forbidden	Requires `mov`
Slice (`T[]`)	Copy `{len, data}` pair	Always allowed
Raw pointer (`*T`)	Copy address	Always allowed

Constraint

A linear value cannot be implicitly copied: transferring it (by assignment, argument, or return) without an explicit `mov` is an implicit move and is *ill-formed* (`[E007]`, § 11.3).

19.7 Object representation

1 The *object representation* of a Lain type is its C99 equivalent as defined in § 17.4. Every object is a sequence of `u8` bytes whose count equals the C99 `sizeof` of the corresponding C type.

2 *Value equality* is defined per type:

- Primitives: bitwise equality of the value representation.
- Structs: field-by-field value equality.
- Slices: equality of both the data pointer and the length.
- Raw pointers: address equality.
- ADTs: tag equality and, when tags match, field-by-field equality of the active variant.

19.8 Alignment and layout

1 Lain inherits ISO/IEC 9899 (C17) alignment and padding rules:

- Struct fields are laid out in declaration order.
- Each field is aligned to its natural alignment (the alignment of its C99 equivalent type).
- The compiler inserts padding between fields as necessary.
- The total size of a struct is rounded up to a multiple of its largest field alignment.

2 Lain does not provide layout control annotations. There is no `#pragma pack` equivalent, no `alignas`, and no `repr(packed)`. The layout is always the platform-default C99 layout.

implementation-defined[3].

[3] Implementation-defined behavior: The exact size and alignment of each type is implementation-defined, following the C99 ABI of the target platform

19.9 ADT and enum layout

1 An enum type is emitted as a C `enum` wrapped in a named struct so that each enum type is nominally distinct in the C translation:

```
typedef enum { Color_Red, Color_Green, Color_Blue } Color_Tag;
typedef struct { Color_Tag tag; } Color;
```

2 An ADT type uses a niche-based encoding whenever a zero-cost representation is derivable. The compiler performs *aggressive multi-slot niche optimization* (D-Niche): it explores the full sentinel space of the payload and packs as many empty variants as fit, without any tag byte. When the pool is insufficient to cover every empty variant, the layout falls back to a tagged union and the compiler emits `[W120]` (best-effort warning; not an error).

```
// Tagged-union fallback (only when no niche is derivable):
typedef struct {
    enum { Shape_Circle_tag, Shape_Rect_tag } tag;
    union {
        struct { float r; } Circle;
        struct { float w; float h; } Rect;
    };
} Shape;
```

3 *Niche encoding (zero-cost).* When the payload of one variant contains a bit pattern that cannot occur in valid values—for instance, `0x0` for a pointer field, or a value outside a VRA-proven range for an integer field—that pattern is reserved to encode an alternative variant. No explicit tag is emitted.

```
// Niche encoding for Option(*File):  size = sizeof(*File)
typedef File *Option_Ptr_File;
//  Some(p)   -> non-null pointer p
//  None      -> 0x0
```

4 The sentinel pool size depends on the target. For pointer payloads, the pool is the zero-page-unmappable region of the target's address space, stepped by the pointer alignment (e.g. 16384 slots for x86_64-linux with alignment 8). For integer payloads, the pool is the type's full range minus any VRA-narrowed sub-range: `type Pct = i32 >= 0 and <= 100` as a payload yields a pool of $\approx 4.3 \times 10^9$ slots (every integer outside $[0, 100]$). For `bool`, the pool is the 254 byte patterns outside $\{0, 1\}$.

5 Niche allocation cascades through nested enum payloads. If `Option(*T)` reserves the value `0` for `None`, a surrounding `Result(Option(*T), Err4)` sees the remaining slots $(8, 16, 24, \ldots)$ and packs its own empty variants into them. Layout of the entire result still occupies one pointer-sized word.

6 *Multi-payload niche.* An enum with exactly two payload variants where one payload (the *primary*) has a non-empty sentinel pool and the other payload (the *secondary*) has a single field whose type is an *enum of pure-empty variants* is encoded by mapping each sub-variant into a sentinel of the primary's pool. For `Result(*T, FileErr)` with `FileErr = enum { NotFound, Permission, OOM }`, the encoding on a host with pointer alignment 8 is `Ok(f)` = any valid pointer, `Err(NotFound)` = sentinel 0, `Err(Permission)` = sentinel 8, `Err(OOM)` = sentinel 16. The secondary constructor emits a stride-multiplication; the case-match discriminator emits a corresponding stride-division for binding-style patterns.

7 The compiler never refuses compilation to satisfy a layout optimization: when the pool is insufficient, `[W120]` is emitted with suggestions (constrain the payload, change to a type with

larger sentinel space, or split the enum), and a tag byte is added. The programmer decides whether to accept the overhead.

8 The CLI flag `-target=<triple>` selects the target's sentinel pool size (host auto-detection is the default). Supported triples include `x86_64-linux-gnu`, `aarch64-linux-gnu`, `x86_64-windows-msvc`, and `cortex-m4-bare` (the last has zero pool: no MMU means address 0 may be a valid vector table entry).

19.10 No garbage collector

1 Lain has no garbage collector. Consequently:

- execution is never interrupted for memory management;
- deallocation occurs at statically deterministic program points;
- no reference counting or heap tracing is performed at runtime.

2 This makes Lain suitable for real-time, embedded, and safety-critical applications where garbage-collection pauses are unacceptable.

19.11 Memory safety in the safe fragment

1 In the *safe fragment* (code outside `unsafe` blocks), the following memory safety properties hold by construction:

Property	Enforcement mechanism
No use-after-free	`[E001]` (use-after-move)
No double-free	`[E002]` (double-consume)
No leak of linear values	`[E003]` (unconsumed at scope exit)
No dangling references	`[E010]` (ref to automatic-duration object)
No out-of-bounds access	`[E085]` (VRA proof failure)
No null-pointer deref	No raw pointers in safe fragment
No data races	No concurrency in current version

Constraint

Inside `unsafe` blocks, pointer operations may violate the above properties if the programmer's safety assertion is incorrect (§ 18.5).

19.12 Compiler-internal arena (informative)

Note

The reference implementation uses a bump-pointer arena internally for all AST nodes, symbol table entries, and diagnostic objects. This arena is an implementation detail; it is never exposed to user code. User programs may implement arena allocators of their own through C interop.

Chapter 20

Standard library

20.1 Overview

1 The Lain standard library is a collection of modules under the `std` namespace (§ 16.6). All standard library modules are written in Lain, using C interop (§ 17) for system-level operations. They are subject to the same ownership, borrow-checking, and purity rules as user code.

2 This clause is normative for the modules listed in Table 20.1. All other modules described in informative documentation are not part of this specification.

Table 20.1: Standard library module index

Module path	Description	Status
`std.c`	C standard library bindings	Normative
`std.io`	Console I/O	Normative
`std.fs`	File system operations	Normative
`std.math`	Pure mathematical functions	Normative

> **Constraint**
>
> A conforming implementation shall provide all modules listed in Table 20.1 at the specified module paths.

20.2 `std.c` — C standard library bindings

20.2.1 Purpose

1 `std.c` provides low-level bindings to selected C standard library functions. It is the foundation on which all other standard library modules that require system calls are built.

20.2.2 Required C headers

2 `std.c` emits the following C include directives into the translation output:

```
c_include "<stdio.h>"
c_include "<stdlib.h>"
```

20.2.3 Opaque types

3 `std.c` declares the following opaque C type:

```
extern type FILE
```

`FILE` is an opaque type representing a C standard I/O stream. It may only be used through pointers and passed to the `extern` functions declared below.

> **Constraint**
>
> `FILE` shall not be used as a value type. A declaration such as `var f FILE` is *ill-formed* (§ 17.8).

20.2.4 Extern declarations

The following C functions are declared in `std.c`:

Declaration	Ownership notes
`extern proc libc_printf(fmt *u8, ...)int`	Shared `fmt`; variadic
`extern proc libc_puts(s *u8)int`	Shared `s`
`extern proc fopen(filename *u8, mode *u8)mov *FILE`	Returns owned handle
`extern proc fclose(stream mov *FILE)int`	Consumes handle
`extern proc fputs(s *u8, stream *FILE)int`	Shared args
`extern proc fgets(s var *u8, n int, stream *FILE)var *u8`	Mutable buffer

> **Note**
>
> `libc_printf` and `libc_puts` are renamed to avoid conflicts with user-defined functions named `printf` and `puts`.

20.3 `std.io` — Console I/O

20.3.1 Dependencies

`std.io` imports `std.c`.

20.3.2 `print`

---

print *(module `std.io`)*

Synopsis: `proc print(s *u8[:0])`
Effect: Writes the null-terminated byte string `s` to standard output. No trailing newline is appended.
Parameter `s`: Shared, null-terminated C string (`*u8[:0]`).

20.3.3 `println`

---

println *(module `std.io`)*

Synopsis: `proc println(s *u8[:0])`
Effect: Writes the null-terminated byte string `s` to standard output, followed by a newline character (`0x0A`).
Parameter `s`: Shared, null-terminated C string (`*u8[:0]`).

20.4 `std.fs` — File system

20.4.1 Dependencies

`std.fs` imports `std.c`.

20.4.2 Type: `File`

`File` is a linear struct that wraps a C `FILE*` handle:

```
type File {
    mov handle *FILE
}
```

Because `handle` is annotated `mov`, `File` is a linear type. It must be consumed exactly once, typically by calling `close_file`.

20.4.3 `open_file`

---

open_file *(module `std.fs`)*

Synopsis: `proc open_file(path *u8[:0], mode *u8[:0])mov File`
Effect: Opens the file at `path` with the access mode `mode` (C `fopen` semantics). Returns an owned `File` value. The caller is responsible for closing the file by consuming the `File`.
Parameter `path`: Shared borrow of a null-terminated path string.
Parameter `mode`: Shared borrow of a mode string (`"r"`, `"w"`, `"a"`, etc.).

> **Note**
>
> If `fopen` returns null, the `handle` field is a null pointer. Subsequent operations on such a file produce C-level undefined behavior. A future version may return `Result(File, int)` instead.

20.4.4 `close_file`

---

close_file *(module `std.fs`)*

Synopsis: `proc close_file(mov {handle} File)`
Effect: Closes the file by calling C `fclose` on the extracted `handle` pointer. Consumes the `File` value.
Parameter: The `File` value is consumed by the call; it shall not be used after this point.

20.4.5 `write_file`

---

write_file *(module `std.fs`)*

Synopsis: `proc write_file(f File, s *u8[:0])`
Effect: Writes the byte sequence `s` to the file `f` using C `fputs`.
Parameter `f`: Shared borrow of the file handle.
Parameter `s`: Shared borrow of the string to write.

Example

```
import std.fs

proc main() int {
    var f = open_file("log.txt", "w")
    defer { close_file(mov f) }
    write_file(f, "Hello, world!\n")
    return 0
}
```

20.5 `std.math` — Mathematical functions

20.5.1 Overview

All functions in `std.math` are declared `func` (pure and terminating). They may be called from any context, including other `func` bodies. `std.math` has no dependencies.

20.5.2 `min`

---

min *(module `std.math`)*

Synopsis: `func min(a int, b int)int`
Returns: The smaller of `a` and `b`. If `a == b`, either may be returned (they are equal).

20.5.3 `max`

---

max *(module `std.math`)*

Synopsis: `func max(a int, b int)int`
Returns: The larger of `a` and `b`.

20.5.4 `abs`

---

abs *(module `std.math`)*

Synopsis: `func abs(x int)int`
Returns: The absolute value of `x`, i.e., `x` if `x >= 0`, `-x` otherwise.

> **Note**
> `abs(INT_MIN)` produces implementation-defined behaviour because `|INT_MIN|` is not representable as `int` on two's complement platforms.

20.5.5 `clamp`

---

clamp *(module `std.math`)*

Synopsis: `func clamp(x int, lo int, hi int)int`
Returns: `x` clamped to the inclusive range `[lo, hi]`, i.e., `max(lo, min(x, hi))`.

> **Constraint**
> Behaviour is **undefined behavior**[a] if `lo > hi`.
>
> [a] Undefined behavior: undefined

20.6 Generic types (informative)

1 This section describes generic type constructors that are part of the standard library but are not normative in the current version of this specification.

20.6.1 `Option(T)`

2 `Option(T)` is a type constructor for optional values:

```
type Option(T type) = type {
    Some { value T }
    None
}
```

3 Usage:

```
type OptionInt = Option(int)
var x = OptionInt.Some(42)
case x {
    Some(v): libc_printf("value: %d\n", v)
    None:    libc_printf("no value\n")
}
```

20.6.2 `Result(T, E)`

4 `Result(T, E)` is a type constructor for fallible computations:

```
type Result(T type, E type) = type {
    Ok  { value T }
    Err { error E }
}
```

5 Usage:

```
type FileResult = Result(File, int)

proc try_open(path *u8[:0]) FileResult {
    var f = open_file(path, "r")
```

```
5       return FileResult.Ok(mov f)
6   }
```

Grammar summary

6 This annex collects all grammar productions defined in this specification. It is normative. Productions are listed in the order they are introduced in the body of the specification. Each section heading identifies the originating clause.

7 The grammar uses an extended BNF notation. `::=` introduces a production. Quoted strings are terminals. Unquoted names are non-terminals. `{ X }` denotes zero or more repetitions. `[ X ]` denotes an optional item. `A | B` denotes alternatives.

§5 Lexical grammar

§5.5 Tokens

```
token ::=
    keyword
  | identifier
  | integer-literal
  | float-literal
  | string-literal
  | char-literal
  | bool-literal
  | operator
  | punctuator
```

§5.6 Keywords

```
keyword ::=
    "func"    | "proc"    | "var"     | "mov"     | "type"
  | "return"  | "if"      | "else"    | "while"
  | "for"     | "in"      | "case"    | "import"  | "use"
  | "extern"  | "c_include" | "unsafe" | "comptime" | "defer"
  | "true"    | "false"   | "and"     | "or"      | "as"
  | "decreasing" | "continue" | "break"
```

§5.7 Identifiers

```
identifier       ::= identifier-start { identifier-continue }
identifier-start ::= letter | "_"
identifier-continue ::= letter | decimal-digit | "_"
letter           ::= "A" .. "Z" | "a" .. "z"
```

§5.8 Literals

```
integer-literal  ::=
    decimal-literal
  | hex-literal
  | binary-literal
  | octal-literal
```

```
decimal-literal  ::= nonzero-digit { decimal-digit | "_" }
                   | "0"
hex-literal      ::= "0x" hex-digit { hex-digit | "_" }
binary-literal   ::= "0b" binary-digit { binary-digit | "_" }
octal-literal    ::= "0o" octal-digit { octal-digit | "_" }

decimal-digit    ::= "0" .. "9"
nonzero-digit    ::= "1" .. "9"
hex-digit        ::= decimal-digit | "a" .. "f" | "A" .. "F"
binary-digit     ::= "0" | "1"
octal-digit      ::= "0" .. "7"

float-literal    ::= decimal-literal "." decimal-literal [ float-exponent ]
float-exponent   ::= ( "e" | "E" ) [ "+" | "-" ] decimal-literal

string-literal   ::= '"' { string-char } '"'
string-char      ::= any-source-char-except-'"'-or-'\' | escape-sequence
escape-sequence  ::= '\' ( '"' | '\' | "n" | "t" | "r" | "0" )

bool-literal     ::= "true" | "false"
```

§5.9 Operators and punctuators

```
operator         ::=
    "+"  | "-"  | "*"  | "/"  | "%"
  | "==" | "!=" | "<"  | ">"  | "<=" | ">="
  | "and"| "or" | "!" | "in"
  | "&"  | "|"  | "^"  | "<<" | ">>"
  | "="  | "+=" | "-=" | "*=" | "/=" | "%="
  | "&=" | "|=" | "^=" | "<<=" | ">>="
  | "->" | "."  | ".."

punctuator       ::=
    "("  | ")"  | "{"  | "}"  | "["  | "]"
  | ":"  | ","  | "@"
```

§6 Program structure grammar

```
program              ::= { top-level-declaration }
top-level-declaration ::=
    import-declaration
  | type-declaration
  | function-declaration
  | procedure-declaration
  | extern-declaration
  | c-include-declaration

import-declaration   ::= "import" qualified-identifier [ "." "{" identifier { "," identifier } "}
type-declaration     ::= "type" identifier [ "(" type-param-list ")" ]
                         ( "=" type | struct-body | enum-body )
type-param-list      ::= type-param { "," type-param }
type-param           ::= identifier "type"
qualified-identifier ::= identifier { "." identifier }
```

§7 Type grammar

```
type             ::=
    primitive-type
  | array-type
  | slice-type
  | pointer-type
  | qualified-type
  | constrained-type
  | user-defined-type

primitive-type   ::=
    "int"   | "u8"    | "u16"   | "u32"   | "u64"   | "usize"
  | "i8"    | "i16"   | "i32"   | "i64"   | "isize"
  | "f32"   | "f64"
  | "bool"  | "str"   | "void"

array-type       ::= type "[" integer-literal "]"
slice-type       ::= [ "*" ] type "[]"                    % fat slice; "*" optional
                   | [ "*" ] type "[" size-constraint "]" % sized fat slice
pointer-type     ::= "*" type                             % raw pointer
                   | "*" type "[" integer-literal "]"     % thin ptr + static length
                   | "*" type "[" ":" sentinel-literal "]" % sentinel-terminated
qualified-type   ::= ownership-qualifier type
ownership-qualifier ::= "var" | "mov"
user-defined-type ::= qualified-identifier
constrained-type ::= type relational-op integer-literal
                     { ( "and" | "or" ) type relational-op integer-literal }
relational-op    ::= ">=" | "<=" | ">" | "<" | "=="
```

§8 Expression grammar

```
expression           ::= assignment-expression

assignment-expression ::=
    logical-or-expression
  | lvalue-expression assignment-operator assignment-expression

assignment-operator  ::=
    "="  | "+=" | "-=" | "*=" | "/=" | "%="
  | "&=" | "|=" | "^=" | "<<=" | ">>="

logical-or-expression ::= logical-and-expression { "or" logical-and-expression }

logical-and-expression ::= in-expression { "and" in-expression }

in-expression        ::= equality-expression { "in" equality-expression }

equality-expression  ::= relational-expression
                         { ( "==" | "!=" ) relational-expression }

relational-expression ::= shift-expression
                          { ( "<" | ">" | "<=" | ">=" ) shift-expression }

shift-expression     ::= additive-expression
                         { ( "<<" | ">>" ) additive-expression }

additive-expression  ::= multiplicative-expression
                         { ( "+" | "-" ) multiplicative-expression }
```

```
multiplicative-expression ::= cast-expression
                              { ( "*" | "/" | "%" ) cast-expression }

cast-expression      ::= unary-expression { "as" type }

unary-expression     ::=
    postfix-expression
  | unary-operator unary-expression

unary-operator       ::= "-" | "!" | "mov" | "var"

postfix-expression   ::=
    primary-expression
  | postfix-expression "." identifier
  | postfix-expression "[" expression "]"
  | postfix-expression "(" [ argument-list ] ")"

argument-list        ::= expression { "," expression }

primary-expression   ::=
    identifier
  | literal
  | "(" expression ")"
  | block-expression
  | if-expression
  | match-expression
  | constructor-expression
  | comptime-expression

literal              ::=
    integer-literal
  | float-literal
  | string-literal
  | bool-literal

block-expression     ::= "{" { statement } expression "}"

if-expression        ::=
    "if" expression block-expression
  [ "else" ( if-expression | block-expression ) ]

match-expression     ::=
    "case" [ "&" ] expression "{"
    { match-arm }
    "}"

match-arm            ::= ( pattern-list | "else" ) ":" arm-body

arm-body             ::= expression newline | block-expression

pattern-list         ::= pattern { "," pattern }

pattern              ::=
    literal
  | literal ".." literal                      % inclusive range pattern
  | string-literal
  | qualified-identifier                       % unit variant
  | qualified-identifier "(" [ binding-list ] ")" % variant with bound fields
```

```
binding-list         ::= identifier { "," identifier }

constructor-expression ::=
    qualified-identifier "{" [ field-init-list ] "}"
  | qualified-identifier "(" [ argument-list ] ")"

field-init-list      ::= identifier ":" expression { "," identifier ":" expression }

comptime-expression  ::= "comptime" block-expression

lvalue-expression    ::=
    identifier
  | postfix-expression "." identifier
  | postfix-expression "[" expression "]"
```

§9 Statement grammar

```
statement            ::=
    block-statement
  | immutable-declaration
  | var-declaration
  | assignment-statement
  | expression-statement
  | if-statement
  | while-statement
  | bounded-while-statement
  | return-statement
  | defer-statement
  | unsafe-block
  | comptime-if-statement

block-statement      ::= "{" { statement } "}"

immutable-declaration ::= identifier [ type ] "=" expression newline

var-declaration      ::= "var" identifier [ type ] "=" expression newline

assignment-statement ::= lvalue-expression assignment-operator expression newline

expression-statement ::= expression newline

if-statement         ::=
    "if" expression block-statement
  { "else" "if" expression block-statement }
  [ "else" block-statement ]

while-statement      ::= "while" expression block-statement

bounded-while-statement ::=
    "while" expression "decreasing" expression block-statement

return-statement     ::= "return" [ expression ] newline

defer-statement      ::= "defer" block-statement

unsafe-block         ::= "unsafe" block-statement
```

```
comptime-if-statement ::=
    "comptime" "if" expression block-statement
  [ "else" block-statement ]
```

§10 Declaration grammar

```
function-declaration  ::=
    { attribute } "func" identifier
    "(" [ param-list ] ")" return-annotation
    block-statement

procedure-declaration ::=
    { attribute } "proc" identifier
    "(" [ param-list ] ")" return-annotation
    block-statement

attribute            ::= "[" identifier [ "(" argument-list ")" ] "]" newline

return-annotation    ::= type [ relational-op integer-literal ]
                       | ( empty )

param-list           ::= param { "," param }
param                ::=
    [ ownership-qualifier ] identifier type              % value parameter
  | identifier "type"                                    % type parameter (generics)
  | [ ownership-qualifier ] "{" identifier { "," identifier } "}" type
                                                         % field-destructuring parameter

struct-body          ::= "{" { field-declaration } "}"
field-declaration    ::= [ "mov" ] identifier type newline

enum-body            ::= "{" { variant-declaration } "}"
variant-declaration  ::=
    identifier newline
  | identifier "{" field-declaration-list "}" newline

field-declaration-list ::= field-declaration { field-declaration }
```

Diagnostic codes

8 This annex is normative. It lists every diagnostic code defined by this specification, the clause that requires the diagnostic, the category (constraint violation, warning, or informational), and a representative message text. A conforming implementation shall issue a diagnostic for every constraint violation in the table below.

9 Diagnostic codes are divided into the following ranges:

Range	Domain
`[E001]`–`[E010]`, `[E016]`–`[E019]`	Ownership and lifetime
`[E011]`–`[E015]`, `[E060]`	Purity, safety, and verification
`[E080]`–`[E082]`	Bounded while verification
`[E083]`–`[E087]`	Array/slice safety and integer boundaries
`[E084]`, `[E101]`–`[E125]`	Modules, attributes, generics, packing, ADT access
`[E100]`	Miscellaneous
`[VRA]`	Value range analysis

Ownership and lifetime diagnostics (`[E001]`–`[E010]`, `[E016]`)

Code	Clause	Message and trigger
`[E001]`	§11.3	*use of moved value 'x'* — Issued when an expression reads or passes a variable whose linear state is CONSUMED.
`[E002]`	§11.3	*value 'x' consumed twice* — Issued when a second `mov` expression is applied to a variable already in state CONSUMED.
`[E003]`	§11.3	*linear value 'x' not consumed before end of scope* — Issued when a `mov`-mode variable exits scope in state UNCONSUMED.
`[E004]`	§11.3	*cannot borrow 'x' as mutable; it is already borrowed* — Issued when a mutable borrow conflicts with an existing borrow (reads or write).
`[E005]`	§11.3	*use of uninitialized variable 'x'* — Issued when a variable, struct field, or array element is read before it has been initialized (definite-assignment analysis).

Code	Clause	Message and trigger
`[E006]`	§11.3	*cannot consume 'x' inside a loop* — Issued when a linear variable or field is consumed inside a loop body that may execute more than once.
`[E007]`	§11.3	*moving linear value 'x' requires an explicit* `mov` — Issued when a linear value is passed to an owned (`mov`) parameter (or otherwise transferred) without the explicit `mov` keyword: an implicit move.
`[E010]`	§19.3	*cannot return reference to local variable 'x'* — Issued when a `return` statement returns a reference whose referent has automatic storage duration.
`[E016]`	§11.3	*linear variable 'x' is used inconsistently in the branches of* stmt — Issued when a linear value is consumed in some control-flow paths of an `if`/`case` but not in others, leaving an ambiguous state at the join point. Field-level inconsistency on linear struct fields is reported with the same code.

Purity, safety, and verification (`[E011]`–`[E015]`)

Code	Clause	Message and trigger
`[E011]`	§12.2	*impure operation in pure function 'f'* — Issued when a `func` body contains a call to a `proc`, an unbounded `while` without a `decreasing` measure, or a direct/mutual recursion without a measure.
`[E012]`	§17.6	*pointer cast outside unsafe block* — Issued when a pointer-to-pointer or integer-to-pointer cast occurs outside an `unsafe` block.
`[E013]`	§6.2	*redeclaration or shadowing of 'x' is forbidden* — Issued for a duplicate parameter name, a local that repeats or shadows an outer binding, or a `main` declared as `func`. (Lain forbids shadowing; use of an *undeclared* identifier is currently undiagnosed — §6.3.)
`[E014]`	§15.4	*non-exhaustive* `case` — Issued when the arms of a `case` do not cover all values of the scrutinee type. (An out-of-bounds *index* is a different code, `[E085]`.)
`[E015]`	§8.3	*division by zero* — Issued when VRA can prove statically that a division or modulo expression has a zero divisor.

Bounded while verification ([E080]–[E082])

Code	Clause	Message and trigger
[E080]	§9.7.1	*termination measure not proven non-negative* — Issued when the compiler cannot prove that the `decreasing` expression is ≥ 0 whenever the loop condition holds.
[E081]	§9.7.1	*cannot extract variables from termination measure* — Issued when the `decreasing` expression is in a form the checker cannot analyze. (A constant or otherwise non-decreasing measure is reported by [E080].)
[E082]	§9.7.1	*termination measure not proven to decrease in loop body* — Issued when the compiler cannot verify that every execution of the loop body strictly decreases the measure by at least 1.

Miscellaneous ([E100])

Code	Clause	Message and trigger
[E100]	§5, §7	*ill-formed token / semantic constraint violation* — A catch-all code used for lexical and syntactic errors without a dedicated code, including: an unexpected token, using `==` on enum types (§7.6), an empty or multi-character character literal, a keyword used as an identifier, and a zero-field array size.

Array/slice safety and integer boundaries ([E083]–[E087])

Code	Clause	Message and trigger
[E083]	§7.5	*linear field 'f' missing `mov` annotation* — Issued when a struct or enum variant field has a linear type but lacks the `mov` keyword. (Q-008 transitivity rule.)
[E085]	§8.12, §8.13	*index or slice endpoint not proven in bounds* — Issued when VRA cannot prove a non-negativity or upper-bound requirement for an array/slice index or sub-slice endpoint.

Code	Clause	Message and trigger
[E086]	§ 13.10	*integer overflow at boundary: range exceeds target type* — Issued when the VRA range of a source value provably exceeds the natural range of the target type at a type boundary (declaration, assignment, call argument, return, struct constructor).
[E087]	§ 8.13, § 13.12	*sized slice constraint violated* — Issued (1) when both endpoints of a sub-slice `arr[a..b]` are integer literals and $a > b$; (2) at a call site where VRA can prove that the argument does not satisfy the formal parameter's sized slice constraint (`T[k]`, `T[>= k]`, `T[> k]`, or `T[ref.len]`).

Value range analysis ([VRA])

Code	Clause	Message and trigger
[VRA]	§ 13	*value range proof required* — Issued when VRA cannot prove that an operation satisfies its required constraint and more specific code is not available. Appears as a diagnostic prefix in messages such as: "`[VRA] cannot prove index in bounds`" and "`[VRA] return value may not satisfy constraint`".

Additional diagnostics ([E008], [E009], [E017]–[E125])

The following diagnostics are emitted by the reference implementation and are normative. Each was verified against the compiler's diagnostic strings during the § 7 reconciliation pass.

Code	Message and trigger
[E008]	cannot move a value that is currently borrowed
[E009]	cannot mutate a field through a raw (shared) pointer — use a `var` borrow
[E017]	passing to a `var` parameter requires explicit `var` at the call site
[E018]	a function can reach its end without returning a value
[E019]	use of a partially-initialized value
[E060]	pointer dereference outside an `unsafe` block
[E063]	a possibly-marker union value used without testing it first
[E064]	a `|`-union cannot be niche-optimized (no spare bit pattern)
[E084]	access to a `[private]` declaration from another module
[E101]	calling a `proc` from a `comptime` context (only `func` is allowed)

Code	Message and trigger
`[E102]`	expected an attribute name after the opening bracket
`[E103]`	unknown attribute
`[E104]`	a type contains itself by value (infinite size)
`[E105]`	identifier is defined in another module — add an `import`
`[E121]`	struct-field packing supports only iN/uN fields totaling ≤ 64 bits
`[E122]`	a function pointer must be initialized with a matching function
`[E123]`	wrong number of arguments in a call through a function pointer
`[E124]`	type application on a non-generic type
`[E125]`	direct ADT field access outside `unsafe` — destructure with `case`

Language design rationale

This annex is informative. It explains the reasoning behind key design decisions in the Lain language. Understanding the rationale helps readers interpret edge cases in the normative text and guides implementors when the specification is ambiguous.

.1 The `func`/`proc` distinction

Decision. Separate pure functions (`func`) from side-effecting procedures (`proc`) at the declaration level.

Rationale.

- *Termination guarantees*: a `func` is guaranteed to terminate (no unbounded `while`, no unchecked recursion). This enables the compiler to reason statically about function completion and to use `func` bodies inside `comptime` expressions.
- *Purity propagation*: the type system propagates purity constraints — a `func` cannot call a `proc`, so pure code stays pure by construction.
- *Optimisation*: the compiler may freely reorder, memoize, or eliminate calls to `func` because they have no observable side effects.
- *Readability*: the declaration keyword tells the reader whether a callable has side effects without inspecting its body.

Alternatives considered. Single keyword with a purity annotation (`func(pure)`) was rejected as less visible. A Haskell-style IO monad was rejected as too complex for a systems language.

.2 Explicit `mov` at call sites

Decision. Require explicit `mov` at call sites when transferring ownership.

Rationale.

- *Readability*: `close_file(mov f)` makes ownership transfer visible at the call site.
- *Intentionality*: accidental ownership transfer is a common source of bugs. Requiring `mov` forces the programmer to acknowledge the transfer.
- *Symmetry*: the annotation appears in both declarations (`mov x T`) and at call sites (`mov x`), creating a uniform language.

Alternatives considered. Rust-style implicit moves were rejected because they are invisible at call sites. A library function `std::move` (C++) was rejected because ownership transfer should be a language primitive, not a library function.

.3 Implicit NLL lifetimes (no annotations)

Decision. Borrows have implicit lifetimes determined by non-lexical lifetime analysis. No explicit lifetime annotations are required.

Rationale.

- *Simplicity*: Rust's lifetime annotations are widely cited as its steepest learning curve. Lain targets programmers who want safety without that complexity.
- *Sufficient expressiveness*: NLL handles the vast majority of real-world borrow patterns.
- *Predictability*: borrows expire at last use — a single, intuitive rule.

Trade-offs. Some programs that Rust can verify with explicit lifetime annotations may be rejected by Lain's more conservative analysis. Higher-order borrowing patterns are constrained.

.4 C99 translation output

Decision. Compile to C99 source code rather than LLVM IR, machine code, or bytecode.

Rationale.

- *Portability*: C99 compilers exist for every platform, from server-class x86 to microcontrollers. Lain inherits this portability for free.
- *Toolchain reuse*: users get GCC/Clang optimisation passes, debuggers, profilers, and sanitisers without custom tooling.
- *Interoperability*: C interop is zero-cost because the output *is* C.
- *Simplicity*: a C emitter is orders of magnitude simpler than an LLVM backend or a native code generator.
- *Auditability*: the generated C code is human-readable.

Trade-offs. Compilation is two-phase (Lain → C → binary). Some optimisations are impossible without direct IR control. The C type system constrains what Lain can express (e.g., no guaranteed tail calls).

.5 No exception mechanism

Decision. Lain has no exception mechanism. All error handling uses return values.

Rationale.

- *Zero overhead*: no unwinding tables, catch tables, or RTTI.
- *Determinism*: error handling follows the same control flow as normal code; no invisible jumps.
- *Embedded compatibility*: stack unwinding is unavailable or unreliable on many embedded platforms.
- *Explicit error paths*: every error is visible in the function signature (via `Result` return types).
- *Compatibility with* `defer`: the `defer` mechanism provides deterministic cleanup without exceptions.

.6 Explicit `unsafe` blocks

Decision. Pointer dereference and direct ADT field access require an explicit `unsafe` block.
Rationale.

- *Auditability*: safety-critical codebases can grep for `unsafe {` to enumerate all potentially dangerous operations.
- *Minimal scope*: Lain's `unsafe` is narrow — it enables only pointer dereference, address-of, casts, direct ADT access, and bounds suppression. Unlike C (where everything is unsafe by default) or some languages (where `unsafe` enables a much larger capability set), Lain's `unsafe` is a surgical escape hatch.
- *Social contract*: the keyword communicates to code reviewers that extra scrutiny is warranted.

.7 Value Range Analysis instead of SMT

Decision. Use interval-based VRA for static verification instead of an SMT solver.
Rationale.

- *Decidability*: VRA always terminates in polynomial time. SMT solving is NP-hard in the general case and may time out.
- *Predictability*: programmers can reason about what the compiler will accept by thinking about integer ranges. SMT solver reasoning is opaque.
- *Simplicity*: the VRA implementation is approximately 600 lines. An SMT integration would be orders of magnitude more complex and would introduce an external dependency.

Trade-offs. VRA is incomplete — it may reject valid programs that an SMT solver could prove correct. Non-linear constraints produce conservatively wide ranges.

.8 Comptime generics instead of `<T>` syntax

Decision. Use compile-time function evaluation with `comptime` parameters instead of traditional generic syntax.
Rationale.

- *Unification*: generics, type aliases, and compile-time assertions all use the same mechanism.
- *Expressiveness*: comptime functions can contain arbitrary logic, enabling type-level programming that traditional generics cannot express.
- *Simplicity*: no angle brackets, no trait bounds, no where clauses. Just functions that take types and return types.
- *Precedent*: inspired by Zig's comptime approach, which has proven ergonomic in practice.

.9 Universal Function Call Syntax (UFCS)

Decision. Support UFCS: `x.f(args)` desugars to `f(x, args)`.
Rationale.

- *No classes needed*: UFCS provides method-like syntax without class hierarchies, virtual dispatch, or inheritance.

- *Chainability*: enables fluent APIs.
- *Discoverability*: IDE auto-completion works naturally — typing a dot after a value shows available operations.
- *Simplicity*: syntactic sugar only; no new concepts, no method tables, no implicit `this` pointer.

.10 Bounded `while` in `func`

1 **Decision.** Allow `while` loops in `func` bodies when a `decreasing` termination measure is supplied.

2 **Rationale.** Without `decreasing`, any `while` loop in a `func` would require a proof of termination that goes beyond what the compiler can verify automatically. Requiring the programmer to supply a *measure* — a non-negative integer expression that strictly decreases on every iteration — is a well-established technique from program verification (*ranking functions*). The compiler's three checks (non-negativity, variable dependency, strict decrease) are sufficient to verify termination without an external solver.

3 This allows algorithms such as binary search and GCD computation to be expressed as `func` without resorting to `proc`.

Comparison with related languages

4 This annex is informative. It places Lain in the context of other systems-programming languages to help readers understand its design choices. Comparisons are accurate as of the date of this specification; language features evolve and readers should verify current status.

.11 Feature matrix

Feature	Lain	Rust	C	Zig	Go
Memory safety	Compile-time	Compile-time	Manual	Runtime+GC	GC
Null safety	Option type	Option type	None	Optional	Nil
Exceptions	None (Result)	None (Result)	errno	Error union	panic/recover
Generics	Comptime	Trait-based	Macros	Comptime	Type params
Lifetime annots.	None (NLL)	Required	N/A	None	N/A
Purity tracking	`func`/`proc`	None	None	None	None
Pattern matching	Exhaustive	Exhaustive	switch	switch	switch
UFCS	Yes	No	No	No	No
Backend	C99	LLVM	Native	LLVM	Go compiler
GC / runtime	None	Minimal	None	Minimal	GC

.12 Lain versus Rust

Similarities

- Ownership-based memory safety with no GC.
- Move semantics with explicit transfer.
- No null pointers (Option type for optional values).
- Pattern matching with exhaustiveness checking.
- No exceptions; Result type for error propagation.
- Zero-cost abstractions.

Differences

Aspect	Lain	Rust
Lifetime annotations	Implicit (NLL only)	Explicit (`'a`, `'static`)
Mutable borrow syntax	`var x` at call site	`&mut x`
Move syntax	Explicit `mov x`	Implicit (default)
Generics	Comptime functions	Trait-based + monomorphization
Purity	`func`/`proc` enforced	Convention only
Async	Not supported	`async`/`await`
Traits / Interfaces	Not supported	Full trait system
Macros	Not supported	`macro_rules!` + proc macros
Backend	C99	LLVM
Learning curve	Lower	Higher

Key insight. Lain trades Rust's expressiveness (explicit lifetime annotations, trait system, async/await) for simplicity. The result is a language that is easier to learn but less expressive for complex borrowing patterns.

.13 Lain versus C

Similarities

- Compiles to native code; no GC.
- Explicit memory management.
- No runtime overhead.
- Systems-level programming: raw pointers accessible in `unsafe`.

Differences

Aspect	Lain	C
Memory safety	Compile-time guarantees	Manual (error-prone)
Type safety	Strong, narrow widening	Weak, implicit conversion
Ownership	Tracked by compiler	Programmer discipline
Bounds checking	Static (VRA)	None
Division by zero	Static detection (E015)	Undefined behavior
Pattern matching	Exhaustive `case`	`switch` (fall-through)
Null safety	Option type	Null everywhere
Strings	Slices with length	Null-terminated arrays
Generics	Comptime functions	Macros / `void*`
Header files	Module system	`#include`

Key insight. Lain provides C-level performance with compile-time safety guarantees. The C99 backend enables seamless interop with existing C libraries while eliminating the most common classes of C undefined behavior from the safe fragment.

.14 Lain versus Zig

Similarities

- Comptime-based generics.
- No hidden control flow; no hidden allocations.
- C interoperability as a first-class concern.
- Explicit error handling with no exceptions.

Differences

Aspect	Lain	Zig
Memory safety	Compile-time ownership	Runtime (debug)
Ownership system	Borrow checker	Manual
Backend	C99	LLVM + self-hosted
Comptime scope	Type-parameter functions	Full expression evaluation
Error handling	Result type + `case`	Error unions + `try`
Allocator model	C interop (malloc/free)	Custom allocators everywhere

Key insight. Both Lain and Zig use comptime for generics. Zig's comptime is more powerful (arbitrary expression evaluation); Lain's ownership system provides stronger compile-time memory safety guarantees than Zig's debug-mode runtime checks.

.15 Lain versus Go

Similarities

- Simple, readable syntax.
- Fast compilation.
- No class hierarchies.

Differences

Aspect	Lain	Go
Memory management	Ownership (no GC)	Garbage collected
Runtime	None	GC + goroutine scheduler
Generics	Comptime functions	Type parameters
Error handling	Result type	Multiple return values
Null safety	Option type	Nil (null)

Aspect	Lain	Go
Concurrency	Not supported	Goroutines + channels
Determinism	Yes (no GC pauses)	GC pauses possible

Key insight. Go achieves simplicity through garbage collection. Lain achieves similar simplicity goals through ownership-based safety, without a runtime — making Lain suitable for embedded and real-time applications where Go's GC is problematic.

.16 Features unique to Lain

The following features distinguish Lain from all languages in the comparison:

Feature	Description
`func`/`proc` split	Compiler-enforced purity with termination guarantees. No other listed language enforces purity at the declaration level.
Caller-site ownership annotations	`mov` and `var` are visible at every call site, making ownership transfer explicit and auditable.
Interval-based VRA	Polynomial-time static verification of array bounds and arithmetic constraints, without an SMT solver.
`in` operator for index constraints	`idx in arr` establishes the bounds guard $0 \leq$ `idx` $<$ `arr.len` in the containing scope.
Bounded `while` with `decreasing`	Allows terminating loops in pure functions via programmer-supplied ranking functions, verified by the compiler.
Non-consuming match (`case &expr`)	Pattern matching that does not consume the scrutinee; the scrutinee is borrowed for the duration of the match body.

Index